

گزارش فاز 1

بخش مدیریت کاربران، احراز هویت و کنترل سطح دسترسی (Users App)

1. طراحی مدل کاربر (users/models.py)

در این پروژه، به منظور پوشش نیازمندی‌های خاص سامانه اداره پلیس، از یک مدل کاربر سفارشی مبتنی بر `AbstractUser` استفاده شده است. این مدل علاوه بر فیلدهای پیش فرض `Django`، شامل فیلدهای زیر می‌باشد:

- کد ملی (`national_id`): به صورت یکتا، جهت شناسایی رسمی افراد
- شماره تماس (`phone_number`): یکتا، برای ارتباط و ورود به سامانه
- نام و نام خانوادگی (`full_name`)
- ایمیل (`email`): یکتا
- تصویر پروفایل/تصویر مزنون (`photo`): جهت استفاده در بخش معرفی مزنونین و گزارش‌های قضایی

همچنین متد `get_roles_display` برای نمایش نقش‌های کاربر (بر اساس گروه‌های `Django`) پیاده‌سازی شده است که در گزارش‌گیری‌ها و پاسخ‌های API مورد استفاده قرار می‌گیرد.

پوشش نیازمندی‌های پروژه:

- فصل 1-4: ثبت نام و ورود کاربران با اطلاعات هویتی کامل
- الزام یکتایی نام کاربری، کد ملی، ایمیل و شماره تماس
- آماده‌سازی زیرساخت برای نمایش تصویر مزنونین در بخش «تحت پیگیری شدید»

امتیازهای پوشش داده شده در چک پوینت:

- طراحی معقول و دقیق مدل های موجودیت ها
- رعایت نیازمندی های ثبت نام و ذخیره اطلاعات هویتی کاربران

2. پیاده سازی احراز هویت چندشناسه ای (users/authentication.py)

به منظور انطباق با نیازمندی پروژه، یک Backend سفارشی برای احراز هویت پیاده سازی شده است که امکان ورود کاربران به سامانه با استفاده از یکی از چهار شناسه زیر را فراهم می کند:

- نام کاربری
- کد ملی
- شماره تماس
- ایمیل

در این پیاده سازی، کاربر بر اساس یکی از این چهار مقدار جستجو شده و سپس رمز عبور با استفاده از متد استاندارد `check_password` بررسی می شود. بدین ترتیب، ورود به سامانه مطابق صورت پروژه و بدون وابستگی به تنها یک شناسه انجام می پذیرد.

پوشش نیازمندی های پروژه:

- فصل 4-1: ورود کاربران با رمز عبور و یکی از شناسه های تعریف شده

امتیازهای پوشش داده شده در چک پوینت:

- پیاده سازی Endpoint های ثبت نام و ورود
- رعایت اصول احراز هویت در Django و DRF

3. سریالایزهای ثبت نام و نمایش کاربران (users/serializers.py)

در این بخش سه سریالایزر اصلی پیاده سازی شده است:

1. RegisterSerializer

مسئول اعتبارسنجی داده های ورودی هنگام ثبت نام و ایجاد کاربر جدید است. در زمان ایجاد کاربر، به صورت پیش فرض نقش «کاربر پایه (Base User)» به وی تخصیص داده می شود.

2. UserSimpleSerializer

برای نمایش اطلاعات خلاصه کاربران (شناسه، نام، ایمیل، نقش ها و آدرس تصویر پروفایل) مورد استفاده قرار می گیرد و در پاسخ های API مربوط به پروفایل و اطلاعات کاربر به کار می رود.

3. GrantSerializer

جهت دریافت داده های لازم برای تخصیص نقش به کاربران توسط مدیر سامانه استفاده شده است.

پوشش نیازمندی‌های پروژه:

- فصل 2-2: تعریف نقش‌ها و کاربر پایه
- فصل 4-1: ثبت نام کاربران و تخصیص نقش پیش فرض
- امکان اعطای نقش توسط مدیر سامانه

امتیازهای پوشش داده شده در چک پوینت:

- وجود Endpoint های مناسب برای ثبت نام
- پیاده سازی کنترل نقش‌ها
- تغییرپذیری نقش‌ها بدون تغییر در منطق اصلی سامانه

4. ویوهای ثبت نام، ورود و مدیریت نقش‌ها (users/views.py)

در این بخش، Endpoint های اصلی مربوط به کاربران پیاده سازی شده اند:

- **RegisterView:** نقش Administrator ثبت نام کاربران جدید. در صورت ثبت نام اولین کاربر سامانه، نقش به صورت خودکار به وی تخصیص داده می شود.
- **CustomAuthToken:** ورود کاربران و دریافت توکن احراز هویت با پشتیبانی از چند شناسه ورودی
- **UserViewSet:** مدیریت کاربران توسط مدیر سامانه شامل:

- مشاهده لیست کاربران
- مشاهده پروفایل کاربر جاری
- مشاهده نقش‌های کاربر
- اعطای نقش به کاربران دیگر

پوشش نیازمندی‌های پروژه:

- فصل 4-1: ثبت نام و ورود
- فصل 2-2: مدیریت نقش‌ها توسط مدیر سامانه
- پیاده سازی نقش مدیر کل سامانه (Administrator)

امتیازهای پوشش داده شده در چک پوینت:

- پیاده سازی Endpoint های مناسب برای روند ثبت نام و ورود
- کنترل سطح دسترسی نقش-پایه
- رعایت اصول REST در طراحی API

5. پیاده سازی مجوزهای دسترسی (users/permissions.py)

برای پیاده‌سازی کنترل دسترسی نقش-پایه و روندی، مجموعه‌ای از Permission Class ها طراحی شده است که نقش‌های مختلف سازمان پلیس را پوشش می‌دهند، از جمله:

- Administrator
- Cadet
- Police Officer / Patrol Officer
- Detective
- Sergeant
- Captain
- Chief
- Coroner
- Judge
- Base User

علاوه بر این، مجوزهای روندی برای سناریوهای زیر پیاده‌سازی شده است:

- بررسی شکایات توسط کارآموز
- بررسی پرونده توسط افسر
- ثبت شواهد
- ویرایش و حذف شواهد
- تأیید شواهد زیستی توسط پزشک قانونی

این ساختار امکان اعمال محدودیت‌های دقیق در سطوح مختلف سیستم را فراهم کرده و تضمین می‌کند که هر نقش تنها به عملیات مجاز خود دسترسی داشته باشد.

پوشش نیازمندی‌های پروژه:

- فصل 3: رده‌های پلیس
- فصل 2-4 و 3-4: تشکیل پرونده و ثبت شواهد
- فصل 2-3: نقش پزشک قانونی در تأیید شواهد زیستی

امتیازهای پوشش داده‌شده در چک‌پوینت:

- بررسی سطح دسترسی و پاسخ درخور در API Call
- پیاده‌سازی کنترل سطح دسترسی نقش-پایه
- پیاده‌سازی Endpoint های روندی مرتبط با ثبت شواهد

6. ایجاد خودکار نقش‌ها در زمان مهاجرت (users/apps.py)

در متد ready اپلیکیشن users، گروه‌های مورد نیاز سامانه (مانند Detective، Sergeant، Captain و Chief و ...) به صورت خودکار پس از اجرای مهاجرت‌ها ایجاد می‌شوند. این کار باعث می‌شود وابستگی به ایجاد دستی نقش‌ها از بین برود و سامانه در هر محیطی به صورت آماده اجرا شود.

پوشش نیازمندی‌های پروژه:

- فصل 2-2: وجود نقش‌های پایه سامانه
- امکان افزودن یا حذف نقش‌ها بدون تغییر در کد

امتیازهای پوشش داده شده در چک پوینت:

- تغییرپذیری نقش‌ها بدون دخالت در کد
- تمیزی معماری و سهولت راه‌اندازی پروژه

7. تست‌های واحد (users/tests.py)

برای اپلیکیشن users مجموعه‌ای از تست‌های واحد طراحی شده است که سناریوهای زیر را پوشش می‌دهد:

- ثبت نام موفق کاربر
- جلوگیری از ثبت نام ناقص
- بررسی یکتایی نام کاربری
- ورود موفق پس از ثبت نام
- جلوگیری از ورود با رمز عبور نادرست
- اعطای نقش به کاربر توسط مدیر سامانه

پوشش نیازمندی‌های پروژه:

- اطمینان از صحت عملکرد روند ثبت نام و ورود
- صحت عملکرد سیستم نقش‌ها

امتیازهای پوشش داده شده در چک پوینت:

- وجود تست در یکی از اپلیکیشن‌های پروژه

گزارش کامل اپلیکیشن Case

(مدیریت شکایت اولیه، گزارش صحنه جرم و تشکیل پرونده نهایی)

1. هدف و جایگاه اپلیکیشن Case در معماری سامانه

اپلیکیشن case مسئول پیاده‌سازی هسته‌ی فرایندهای عملیاتی سیستم اداره پلیس است و سه مسیر اصلی تشکیل پرونده را پوشش می‌دهد:

1. مسیر شکایت مردمی (Register Complain)

دریافت شکایت از شهروندان و عبور آن از چند مرحله بررسی (کارآموز ← افسر پلیس) تا تشکیل پرونده رسمی.

2. مسیر گزارش صحنه جرم توسط مأمور (Crime Scene Report)

ثبت گزارش میدانی توسط نیروهای پلیس و تأیید آن توسط مافوق برای تشکیل پرونده.

3. مدیریت پرونده نهایی (Case)

ایجاد پرونده رسمی، تخصیص شماره پرونده، افزودن مظنون و آغاز فرآیند تحقیقات توسط کارآگاه.

این اپلیکیشن عملاً ستون فقرات جریان داده از «ثبت حادثه» تا «تشکیل پرونده و شروع تحقیقات» محسوب می‌شود.

2. طراحی مدل‌های دامنه (Domain Models)

2-1. مدل شکایت اولیه (RegisterComplain)

مدل RegisterComplain نماینده‌ی شکایت ثبت‌شده توسط شهروند (Base User) است و شامل اطلاعات زیر می‌باشد:

- اطلاعات شاکی اصلی (creator)
- عنوان و شرح کامل شکایت
- زمان و مکان تقریبی وقوع حادثه
- نوع جرم (CrimeType: سطح ۱ تا بحرانی)
- وضعیت شکایت (ComplainStatus)
- شمارنده دفعات بازگشت جهت اصلاح (revision_count) با سقف مجاز (max_revisions)

وضعیت‌های شکایت (State Machine):

- DRAFT : پیش‌نویس در اختیار شاکی
- PENDING_CADET : ارسال‌شده برای بررسی کارآموز
- RETURNED_TO_COMPLAINANT : بازگشت برای اصلاح
- PENDING_OFFICER : در انتظار بررسی افسر پلیس
- APPROVED : تأیید نهایی و تشکیل پرونده
- CANCELLED : لغو خودکار پس از رسیدن به سقف اصلاحات

منطق کنترلی مانند can_submit () و can_be_edited_by_complainant () تضمین می‌کند که شاکی فقط در وضعیت‌های مجاز امکان ویرایش یا ارسال مجدد داشته باشد.

پوشش نیازمندی‌ها:

- تعریف چرخه عمر شکایت
- جلوگیری از سوءاستفاده با محدود کردن دفعات بازگشت
- امکان ثبت نوع جرم و استفاده از آن در تشکیل پرونده نهایی

2-2. مدل شاکیان اضافی (Complainant)

مدل Complainant امکان ثبت چند شاکی برای یک حادثه را فراهم می‌کند و در دو مسیر استفاده می‌شود:

- متصل به شکایت اولیه (complain) (case)
- متصل به پرونده نهایی (case)

هر شاکی وضعیت جداگانه دارد (در انتظار تأیید، تأییدشده، ردشده) و از ثبت تکراری یک کاربر برای یک شکایت یا پرونده جلوگیری شده است (unique_together).

پوشش نیازمندی‌ها:

- پشتیبانی از چند شاکی برای یک پرونده
- کنترل صحت داده و جلوگیری از ثبت تکراری

2-3. مدل بررسی شکایت (ComplainReview)

هر مرحله بررسی شکایت توسط کارآموز یا افسر پلیس در مدل ComplainReview ثبت می‌شود که شامل:

- فرد بررسی‌کننده
- پیام یا توضیح
- وضعیت مقصد شکایت پس از بررسی
- ثبت تاریخچه بررسی‌ها برای ردگیری تصمیم‌ها

این ساختار، شفافیت و قابلیت گزارش‌گیری از روند تصمیم‌گیری‌ها را فراهم می‌کند.

2-4. مدل گزارش صحنه جرم (CrimeSceneReport)

مدل CrimeSceneReport برای ثبت گزارش‌های میدانی مأمورین طراحی شده است و شامل:

- گزارش‌دهنده (مأمور پلیس)
 - زمان و مکان وقوع جرم
 - شرح صحنه
 - نوع جرم
 - وضعیت گزارش (پیش‌نویس، در انتظار تأیید مافوق، تأییدشده)
 - مافوق تأییدکننده و زمان تأیید
- پس از تأیید گزارش، به صورت خودکار یک Case مرتبط ایجاد می‌شود.

2-5. مدل شاهد صحنه جرم (CrimeSceneWitness)

این مدل اطلاعات هویتی و خلاصه اظهارات شاهدان را ذخیره می‌کند و به هر گزارش صحنه جرم قابل اتصال است. این بخش زیرساخت ثبت داده‌های اولیه برای فرآیند تحقیقات بعدی را فراهم می‌سازد.

2-6. مدل پرونده نهایی (Case)

مدل Case خروجی نهایی دو مسیر قبلی است:

- یا از طریق شکایت مردمی
- یا از طریق گزارش صحنه جرم

ویژگی‌های کلیدی:

- ایجاد شماره پرونده یکتا به صورت خودکار بر اساس سال و ترتیب ثبت
- وضعیت پرونده (OPEN / CLOSED)
- ارتباط با شکایان و گزارش‌ها
- مبنای شروع فرآیند تحقیقات (Detection)

3. پیاده‌سازی API و جریان‌های کاری (Views)

3-1. مدیریت شکایت اولیه (RegisterComplainViewSet)

جریان کاری شکایت به صورت چندمرحله‌ای پیاده‌سازی شده است:

1. ایجاد شکایت (Base User)
2. ارسال به کارآموز (submit)
3. بررسی کارآموز (cadet_review)

- تأیید → ارسال به افسر
- بازگشت → اصلاح توسط شاکی (با محدودیت دفعات)
- 4. بررسی افسر پلیس (officer_review)
- تأیید → تشکیل پرونده (ایجاد Case)
- بازگشت → برگشت به مرحله کارآموز

در هر مرحله، کنترل سطح دسترسی بر اساس نقش (Base User، Cadet، Police Officer) اعمال شده است.

ویژگی مهم کنترلی:

پس از رسیدن تعداد بازگشت‌ها به سقف تعریف‌شده، شکایت به صورت خودکار در وضعیت CANCELLED قرار می‌گیرد.

3-2. مدیریت گزارش صحنه جرم (CrimeSceneReportViewSet)

مأمور پلیس (غیر کارآموز) می‌تواند گزارش صحنه جرم ثبت کند. پس از ثبت:

- گزارش به مافوق ارسال می‌شود (submit)
 - مافوق آن را تأیید می‌کند (approve) و پرونده تشکیل می‌شود
 - استثنا: اگر گزارش‌دهنده Chief باشد، تأیید مستقیم و بدون نیاز به مرحله مافوق انجام می‌شود
- همچنین امکان ثبت شاهدان و افزودن شاکی به پرونده پس از تشکیل Case فراهم شده است.

3-3. مدیریت پرونده نهایی (CaseViewSet)

برای پرونده‌ها قابلیت‌های زیر پیاده‌سازی شده است:

- مشاهده لیست و جزئیات پرونده‌ها
- افزودن مظنون به پرونده (در وضعیت OPEN)
- شروع فرآیند تحقیقات توسط کارآگاه (start_detection)
- ایجاد DetectionBoard و Detection
- تعیین Sergeant ناظر بر تحقیقات

این بخش نقطه اتصال اپلیکیشن case با اپلیکیشن detection محسوب می‌شود.

3-4. ارائه آمار مدیریتی (StatsView)

آماری برای داشبورد مدیریتی طراحی شده است که موارد زیر را ارائه می‌دهد Endpoint

- تعداد کل پرونده‌های مختومه
- تعداد کارکنان سامانه (غیر از کاربران عادی)
- تعداد پرونده‌های فعال

این بخش برای نمایش وضعیت کلی سازمان در پنل مدیریت کاربرد دارد.

4. طراحی سریالایزرها و غنای خروجی API

در سریالایزرها تلاش شده خروجی‌ها غنی و مناسب مصرف فرانت‌اند باشند:

- نمایش نام کامل کاربران
- نمایش تعداد شاکیان و شاهدان
- نمایش آخرین نظر ثبت‌شده در بررسی شکایت
- محاسبه تعداد دفعات باقی‌مانده برای اصلاح شکایت

این طراحی باعث کاهش تعداد درخواست‌های لازم از سمت کلاینت می‌شود و تجربه کاربری را بهبود می‌دهد.

5. تست‌های واحد (Unit Tests)

برای اپلیکیشن case مجموعه‌ای از تست‌های واحد طراحی شده که سناریوهای کلیدی را پوشش می‌دهند:

- امکان ثبت شکایت توسط کاربر عادی
- جلوگیری از ثبت شکایت بدون احراز هویت
- بررسی جریان بازگشت شکایت توسط کارآموز
- تشکیل پرونده پس از تأیید افسر پلیس
- لغو خودکار شکایت پس از بیش از سه بار بازگشت
- امکان ثبت گزارش صحنه جرم توسط مأمور

گزارش فنی مازول مدیریت شواهد (Evidence Management)

1. هدف و جایگاه مازول در سامانه

ماژول «مدیریت شواهد» یکی از اجزای هسته‌ای سامانه مدیریت پرونده‌های جنایی است و وظیفه‌ی ثبت، نگهداری، بازیابی، و پردازش انواع شواهد مرتبط با یک پرونده را بر عهده دارد. این ماژول به‌صورت کامل از طریق API‌های REST در اختیار سایر بخش‌های سامانه (فرانت‌اند، بکل مدیریت و ماژول حل پرونده) قرار می‌گیرد.

اهداف اصلی این ماژول عبارت‌اند از:

- پشتیبانی از انواع مختلف شواهد با ساختار داده‌ای منعطف
- تضمین صحت داده‌ها از طریق اعتبارسنجی در سطح مدل و سرالایزر
- کنترل دسترسی مبتنی بر نقش کاربران (Role-Based Access Control)
- امکان الصاق فایل‌های رسانه‌ای به شواهد با محدودیت نوع رسانه
- فراهم‌سازی مستندات API از طریق Swagger

2. طراحی مدل داده (Models)

2.1 مدل پایه Evidence

مدل Evidence به‌عنوان کلاس پایه برای تمامی انواع شواهد تعریف شده است و شامل اطلاعات مشترک زیر است:

- شناسه یکتا از نوع UUID
- ارتباط با پرونده (Case)
- دسته‌بندی شواهد (category)
- عنوان و توضیحات
- کاربر ثبت‌کننده و زمان ثبت
- زمان آخرین به‌روزرسانی

این طراحی باعث شده است تمامی شواهد، صرف‌نظر از نوع آن‌ها، در یک ساختار یکپارچه مدیریت شوند و امکان مرتب‌سازی و گزارش‌گیری فراهم گردد.

2.2 انواع شواهد (Inheritance)

برای پوشش نیازهای مختلف سیستم، هر نوع شاهد به‌صورت یک زیرکلاس از Evidence پیاده‌سازی شده است:

- **استشهاد شاهدان (WitnessStatement):** شامل اطلاعات هویتی شاهد، شماره تماس و زمان اخذ اظهارات. این نوع شواهد امکان الصاق تصویر، ویدئو و فایل صوتی را دارد.
- **شواهد زیستی و پزشکی (BiologicalEvidence):** شامل زمان و محل جمع‌آوری نمونه، نتایج بررسی پزشکی قانونی و نتیجه تطبیق با بانک‌های داده

هویتی. این نوع شاهد دارای فرآیند «تأیید» توسط پزشک قانونی یا مقام مجاز است.

- **شواهد وسیله نقلیه (VehicleEvidence):**

شامل مدل خودرو، رنگ و یکی از دو مشخصه‌ی پلاک یا شماره شناسی. اعتبارسنجی تضمین می‌کند که حداقل یکی از این دو مقدار وارد شود و ورود همزمان هر دو مجاز نیست.

- **مدارک شناسایی (IdentityDocumentEvidence):**

برای مدارک یافت‌شده از نوع شناسنامه، کارت شناسایی و موارد مشابه طراحی شده و با استفاده از مدل کمکی IdentityDocumentField امکان ذخیره اطلاعات کلید-مقدار دلخواه (به صورت منعطف) فراهم شده است.

- **سایر شواهد (OtherEvidence):**

برای موارد متفرقه که در دسته‌بندی‌های قبلی نمی‌گنجند و تنها شامل عنوان و توضیحات هستند.

2.3 فایل‌های ضمیمه (EvidenceMedia)

برای هر شاهد امکان الصاق چندین فایل رسانه‌ای (تصویر، ویدئو، صوت، سند) وجود دارد. در سطح مدل، با استفاده از متد `allowed_media_types` () برای هر نوع شاهد، نوع رسانه‌های مجاز کنترل می‌شود. همچنین متد `clean` () از ثبت رسانه‌های نامعتبر جلوگیری می‌کند.

3. لایه سریالایزر (Serializers)

3.1 سریالایزر پایه

تمامی فیلدهای مشترک شواهد را برمی‌گرداند و اطلاعات تکمیلی مانند نام `EvidenceBaseSerializer` کاربری و نام کامل ثبت‌کننده و همچنین لیست رسانه‌های ضمیمه را به صورت فقط خواندنی ارائه می‌دهد.

3.2 سریالایزرهای تخصصی

برای هر زیرنوع شاهد، یک سریالایزر اختصاصی پیاده‌سازی شده است که علاوه بر فیلدهای پایه، فیلدهای تخصصی همان نوع شاهد را نیز برمی‌گرداند. این تفکیک باعث شفافیت ساختار API و سهولت مصرف آن توسط کلاینت می‌شود.

3.3 سریالایزر چندریختی (Polymorphic)

برای عملیات لیست و جزئیات، از `EvidencePolymorphicSerializer` استفاده شده است که بر اساس مقدار فیلد `category`، به صورت پویا سریالایزر مناسب را انتخاب می‌کند. این رویکرد امکان ارائه‌ی پاسخ یکپارچه و در عین حال دقیق را فراهم می‌کند.

3.4 سریالایزر ایجاد شواهد

برای ایجاد شواهد جدید طراحی شده و وظیفه‌ی اعتبارسنجی ورودی‌ها `EvidenceCreateSerializer` بر اساس نوع شاهد را بر عهده دارد.

در این بخش:

- فیلدهای اجباری هر نوع شاهد بررسی می‌شوند.
 - منطق ایجاد شیء نهایی به صورت متمرکز در متد `create` پیاده‌سازی شده است.
 - ایجاد رسانه‌های ضمیمه عمداً به لایه View منتقل شده تا کنترل بهتری بر آپلود فایل‌ها وجود داشته باشد.
-

4. لایه ViewSet و APIها

4.1 EvidenceViewSet

این ViewSet تمامی عملیات CRUD مربوط به شواهد را پوشش می‌دهد:

- دریافت لیست شواهد (با قابلیت فیلتر بر اساس شناسه پرونده)
- مشاهده جزئیات هر شاهد
- ایجاد شاهد جدید
- ویرایش و حذف شاهد

در متد `get_permissions`، دسترسی‌ها به صورت پویا و بر اساس نوع عملیات کنترل شده است (مثلاً فقط کاربران دارای نقش مجاز امکان ثبت یا حذف شاهد را دارند).

4.2 ایجاد شاهد همراه با رسانه

در متد `create`:

- ابتدا داده‌ها اعتبارسنجی می‌شوند.
- شیء شاهد با ثبت خودکار کاربر ایجادکننده ذخیره می‌شود.
- فایل‌های رسانه‌ای ارسال شده به صورت همزمان پردازش شده و نوع رسانه به صورت خودکار از روی پسوند فایل تشخیص داده می‌شود.
- پیش از ذخیره هر فایل، تطابق نوع رسانه با دسته شاهد بررسی می‌شود.

4.3 افزودن رسانه به شاهد موجود

اکشن سفارشی `add_media` امکان افزودن فایل جدید به یک شاهد ثبت شده را فراهم می‌کند. این عملیات نیز دارای کنترل دسترسی اختصاصی بوده و اعتبار نوع رسانه پیش از ذخیره بررسی می‌شود.

4.4 تأیید شواهد زیستی

اکشن سفارشی `verify_biological` برای ثبت نتایج بررسی پزشکی قانونی و تطبیق با بانک داده طراحی شده است. این عملیات فقط برای شواهد زیستی فعال است و اطلاعات تأییدکننده و زمان تأیید به صورت خودکار ثبت می‌شود.

5. مستندسازی API

برای تمامی endpointهای اصلی و اکشنهای سفارشی، از `swagger_auto_schema` استفاده شده است تا مستندات دقیق شامل:

- ساختار درخواست
 - ساختار پاسخ
 - مقادیر مجاز برای فیلدها
- در رابط Swagger نمایش داده شود. این موضوع فرآیند تست و توسعه فرانت اند را تسهیل می کند.
-

6. ارتباط با نیازمندیهای پروژه و چکپوینت

پوشش نیازمندیها:

- ثبت انواع شواهد (زیستی، استنشهاد، خودرو، مدارک شناسایی، سایر)
- الصاق فایل های رسانه ای به شواهد
- فرآیند تأیید پزشکی قانونی برای شواهد زیستی
- کنترل دسترسی مبتنی بر نقش

چکپوینت اول پروژه:

- طراحی مدل های داده ای کامل و توسعه پذیر
 - پیاده سازی CRUD API برای شواهد
 - رعایت اصول REST و جداسازی مسئولیت ها
 - مستندسازی API با Swagger
-

گزارش فنی ماژول Detection (مدیریت فرآیند کشف و پیشنهاد مظنونان)

1. هدف و دامنه ماژول

ماژول Detection به عنوان هسته ی عملیاتی فرآیند کشف جرم در سامانه عمل می کند و وظیفه ی مدیریت تخته کارآگاه (Detection Board)، سرخ ها (Leads)، ارتباط میان سرخ ها (Yarns) و فرآیند پیشنهاد مظنونان به گروه بان را بر عهده دارد. این ماژول نقش پل ارتباطی میان ماژول «شواهد» و ماژول «بازجویی» را ایفا می کند و تضمین می نماید که فرآیند شناسایی مظنونان به صورت کنترل شده، مستند و قابل پیگیری انجام شود.

2. ساختار مدل‌ها و نقش آن‌ها

2-1. DetectionBoard (تخته کشف)

- هر پرونده دارای یک تخته کشف اختصاصی است که به یک کارآگاه منتسب می‌شود.
- تخته کشف محیطی بصری برای ثبت و سامان‌دهی سرنخ‌ها (اعم از مبتنی بر شواهد یا یادداشت‌های تحلیلی کارآگاه) فراهم می‌کند.
- ارتباط یک‌به‌یک میان Detection و DetectionBoard برقرار شده است تا هر پرونده تنها یک تخته رسمی داشته باشد.

2-2. Lead (سرنخ)

- هر سرنخ می‌تواند از دو نوع باشد:
- سرنخ مبتنی بر مدرک (Evidence Lead)
- سرنخ مبتنی بر یادداشت تحلیلی کارآگاه (Note Lead)
- برای هر سرنخ، مختصات مکانی نسبی (position_x و position_y) در بازه 0 تا 1 ذخیره می‌شود تا در رابط کاربری به صورت گرافیکی روی تخته نمایش داده شود.
- قیود اعتبارسنجی تضمین می‌کند که:
- سرنخ مبتنی بر مدرک، الزاماً دارای مدرک مرتبط باشد و محتوای متنی نداشته باشد.
- سرنخ مبتنی بر یادداشت، الزاماً دارای متن باشد و مدرک نداشته باشد.

2-3. Yarn (ارتباط میان سرنخ‌ها)

- نمایانگر یک ارتباط منطقی یا تحلیلی میان دو سرنخ در یک تخته واحد است Yarn.
- اعتبارسنجی در سطح مدل و Serializer تضمین می‌کند که:
- دو سرنخ متصل‌شده حتماً متعلق به یک تخته باشند.
- اتصال یک سرنخ به خودش مجاز نباشد.

2-4. Detection (فرآیند کشف)

- این مدل ارتباط میان پرونده، کارآگاه و گروه‌بان را مدیریت می‌کند.
- هر Detection شامل:
- یک کارآگاه مسئول
- یک گروه‌بان ناظر
- یک تخته کشف اختصاصی
- کنترل سطح دسترسی تضمین می‌کند که نقش‌های کارآگاه و گروه‌بان مطابق با گروه کاربری آن‌ها باشد.

2-5. SuspectsSuggested (پیشنهاد مظنونان)

- این مدل پیشنهاد رسمی کارآگاه به گروه‌بان را ثبت می‌کند.

- وضعیت‌ها:
 - در انتظار بررسی (PENDING)
 - تأیید شده (CONFIRMED)
 - رد شده (REJECTED)
 - هر پیشنهاد شامل:
 - فهرست کاربران مظنون
 - دلایل مستند کارآگاه
 - بازخورد گروه‌بان در صورت رد شدن
 - قیود اعتبارسنجی تضمین می‌کند که:
 - ارائه دلایل برای پیشنهاد الزامی است.
 - در صورت رد پیشنهاد، ثبت بازخورد گروه‌بان اجباری باشد.
-

3. لایه API و کنترل دسترسی

3-1. DetectionBoardViewSet

- امکان مشاهده تخته‌های متعلق به کارآگاه جاری.
- اکشن‌های اختصاصی:
 - دریافت تمام سرخ‌های یک تخته : leads
 - دریافت تمام ارتباطات میان سرخ‌ها : yarns
 - دریافت تمام پیشنهادهاى مظنونان برای پرونده مربوطه : suggestions
 - ارسال رسمی پیشنهاد مظنونان برای بررسی گروه‌بان : submit_suspects
 - دریافت پیام‌های رد پیشنهاد (به صورت اعلان برای کارآگاه) : messages

کنترل‌های کلیدی:

- هر کارآگاه فقط به تخته‌های متعلق به خود دسترسی دارد.
- در هر زمان تنها یک پیشنهاد در وضعیت «در انتظار بررسی» برای هر پرونده مجاز است.
- ارسال پیشنهاد جدید در صورت وجود پیشنهاد در انتظار بررسی مسدود می‌شود.

3-2. LeadViewSet

- پیاده‌سازی CRUD برای سرخ‌ها.
- تضمین می‌شود که سرخ فقط روی تخته‌ای ایجاد شود که متعلق به کارآگاه جاری است.

3-3. YarnViewSet

- پیاده‌سازی CRUD برای ارتباط سرخ‌ها.
- بررسی می‌شود که هر دو سرخ متعلق به یک تخته و آن تخته متعلق به کارآگاه جاری باشد.

3-4. SuspectsSuggestionViewSet

- مشاهده وضعیت پیشنهادها توسط کارآگاه (فقط خواندنی).
- اکشن status جهت بررسی آخرین وضعیت و بازخورد احتمالی گروهیان.

3-5. SergeantSuggestionViewSet

- ویژه گروهیان جهت بررسی پیشنهادها.
- اکشن‌ها:
- pending : دریافت پیشنهادهای در انتظار بررسی
- approve : تأیید پیشنهاد و ایجاد مظنون و فرآیند بازجویی
- reject : رد پیشنهاد همراه با بازخورد رسمی

4. یکپارچگی با مازول بازجویی (Interrogation)

در صورت تأیید پیشنهاد مظنونان توسط گروهیان:

- برای هر کاربر مظنون، یک شیء Suspect در مازول بازجویی ایجاد می‌شود.
- یک فرآیند Interrogation برای هر مظنون ایجاد می‌گردد و وضعیت اولیه آن در حالت «در انتظار امتیازدهی» قرار می‌گیرد.
- در پرونده‌های با درجه اهمیت بحرانی (CRITICAL)، منطق سیستمی بررسی می‌کند که آیا تأیید مقام بالاتر (مثلاً رئیس) نیز مورد نیاز است یا خیر.

5. سناریوی کامل گردش کار (Workflow)

1. کارآگاه تخته کشف را مشاهده می‌کند.
2. سرنخ‌های مبتنی بر شواهد یا یادداشت تحلیلی ثبت می‌نماید.
3. میان سرنخ‌ها ارتباط منطقی (Yarn) ایجاد می‌کند.
4. پس از تحلیل، مظنونان پیشنهادی را به گروهیان ارسال می‌کند.
5. گروهیان پیشنهاد را بررسی نموده و آن را تأیید یا رد می‌کند:
 - در صورت تأیید: ایجاد مظنون و شروع بازجویی
 - در صورت رد: ارسال بازخورد رسمی به کارآگاه
6. کارآگاه وضعیت پیشنهاد و پیام‌های رد را از طریق API دریافت می‌کند.

6. پوشش تست و تضمین کیفیت

برای این مازول مجموعه‌ای از تست‌های یکپارچه پیاده‌سازی شده است که موارد زیر را پوشش می‌دهد:

- محدودسازی دسترسی کارآگاه به تخته‌های خود
- اعتبارسنجی ایجاد سرخ معتبر و نامعتبر
- ایجاد ارتباط میان سرخ‌ها و جلوگیری از ارتباط غیرمجاز
- جلوگیری از ارسال هم‌زمان چند پیشنهاد در وضعیت در انتظار بررسی
- فرآیند کامل تأیید/رد پیشنهاد توسط گروه‌بان
- یکپارچگی با ایجاد مظنون و بازجویی در مازول بازجویی
- کنترل دسترسی کاربران غیرمجاز

گزارش فنی مازول Interrogation (شناسایی مظنونین، بازجویی، محاکمه و افراد تحت پیگیری شدید)

1. هدف و دامنه مازول

مازول Interrogation مسئول پیاده‌سازی فرآیندهای زیر در سامانه مدیریت پرونده‌های پلیس است:

- ثبت و مدیریت مظنونین هر پرونده
- انجام فرآیند بازجویی توسط کارآگاه و گروه‌بان
- ثبت امتیاز احتمال گناهکاری مظنونین
- ثبت نظر نهایی کاپیتان و در صورت نیاز تأیید رئیس پلیس
- ایجاد پرونده قضایی و ارجاع به دادگاه
- ثبت رأی نهایی قاضی و مجازات
- محاسبه و نمایش مظنونین و مجرمان «تحت پیگیری شدید» و پاداش آن‌ها

این مازول پیاده‌سازی مستقیم نیازمندی‌های فصل‌های زیر از صورت پروژه را پوشش می‌دهد:

- شناسایی مظنونین و بازجویی
- محاکمه
- وضعیت مظنونین (تحت تعقیب و تحت پیگیری شدید)
- پاداش (نمایش عمومی لیست افراد تحت پیگیری شدید و مبلغ جایزه)

2. طراحی مدل‌های داده (Models)

2-1. مدل Suspect (مظنون)

این مدل نمایانگر ارتباط یک شخص با یک پرونده به عنوان مظنون است.

ویژگی‌ها:

- اتصال هر مظنون به یک کاربر (person) و یک پرونده (case)
- جلوگیری از ثبت تکراری یک شخص به عنوان مظنون در یک پرونده (unique_together)
- نگهداری وضعیت مظنون:
- WANTED (تحت تعقیب)
- MOST_WANTED (تحت پیگیری شدید)
- CAPTURED (دستگیر شده)
- ثبت زمان اضافه شدن مظنون به پرونده

پوشش نیازمندی پروژه:

- پیاده‌سازی وضعیت مظنونین
- پشتیبانی از لیست افراد تحت پیگیری شدید
- نگهداری سابقه مظنون بودن یک شخص در چند پرونده مختلف

2-2. مدل Interrogation (بازجویی)

این مدل فرآیند بازجویی از یک مظنون را توسط دو نقش رسمی ثبت می‌کند:

- گروه‌بان (Sergeant)
- کارآگاه (Detective)

قوانین و قيود مهم:

- گروه‌بان و کارآگاه نمی‌توانند یک نفر باشند.
- هر نقش باید عضو گروه کاربری مربوط به خود باشد.
- امتیازدهی هر دو نقش عددی بین 1 تا 10 است.
- وضعیت فرآیند بازجویی با وضعیت‌های زیر مدیریت می‌شود:
- در انتظار ثبت امتیازها
- ثبت کامل امتیازها
- در انتظار تصمیم کاپیتان
- تصمیم‌گیری کاپیتان
- در انتظار تأیید رئیس پلیس
- تکمیل فرآیند

پوشش نیازمندی پروژه:

- پیاده‌سازی کامل مرحله «شناسایی مظنونین و بازجویی»
- ثبت امتیاز احتمال گناهکاری توسط کارآگاه و گروه‌بان

- پشتیبانی از فرآیند تصمیم‌گیری سلسله‌مراتبی (کاپیتان و رئیس پلیس)
-

2-3. مدل Court (دادگاه)

این مدل نمایانگر ارجاع رسمی پرونده به قوه قضاییه است.

ویژگی‌ها:

- ارتباط یک‌به‌یک با یک بازجویی
- تخصیص قاضی به پرونده
- ثبت رأی نهایی (GUILTY یا NOT_GUILTY)
- ثبت تاریخ صدور رأی

پوشش نیازمندی پروژه:

- پیاده‌سازی کامل مرحله «محاكمه»
 - امکان مشاهده کلیه اطلاعات پرونده توسط قاضی
-

2-4. مدل Punishment (مجازات)

این مدل برای ثبت مجازات در صورت محکومیت مظنون استفاده می‌شود.

ویژگی‌ها:

- ثبت مبلغ جریمه
- مدت زمان زندان
- تبعید (در صورت وجود)

پوشش نیازمندی پروژه:

- ثبت عنوان و توضیحات مجازات مطابق با صورت پروژه
 - امکان ثبت چند مجازات برای یک دادگاه
-

3. Views و Endpoints ها و روندهای پیاده‌سازی شده API

3-1. مدیریت مظنونین (SuspectViewSet)

قابلیت‌ها:

- مشاهده لیست مظنونین هر پرونده
- جلوگیری از حذف مظنون در صورت وجود بازجویی ثبت شده
- آغاز بازجویی جدید برای مظنون (interrogate)
- دریافت خلاصه وضعیت مظنون شامل:
 - تعداد بازجویی‌ها
 - میانگین امتیاز گروه‌بان
 - میانگین امتیاز کارآگاه
- دریافت تاریخچه بازجویی‌های هر مظنون

سطح دسترسی:

- فقط کاربران غیر Base User مجاز به استفاده از این API هستند.

3-2. مدیریت بازجویی‌ها (InterrogationViewSet)

قابلیت‌ها:

- ثبت امتیاز گروه‌بان
- ثبت امتیاز کارآگاه
- ثبت نظر و رأی کاپیتان
- تأیید یا رد رأی کاپیتان توسط رئیس پلیس در جرائم بحرانی
- فیلتر بازجویی‌ها بر اساس نقش کاربر (کارآگاه فقط موارد مربوط به خود را می‌بیند و ...)

پوشش روند پروژه:

- پیاده‌سازی کامل فرآیند مرحله 4-5 پروژه (شناسایی مظنونین و بازجویی)
- اعمال کنترل سطح دسترسی نقش‌محور (Role-Based Access Control)

3-3. صفحه افراد تحت پیگیری شدید و پاداش (most_wanted_view)

این Endpoint عمومی، لیست افراد تحت پیگیری شدید را همراه با رتبه‌بندی و مبلغ پاداش ارائه می‌دهد.

فرمول پیاده‌سازی شده مطابق صورت پروژه:

- رتبه‌بندی:

$$\text{ranking} = \max(L_j) \times \max(D_i)$$

- پاداش:

پوشش نیازمندی پروژه:

- پیاده سازی کامل فصل 4-7 (وضعیت مظنونین)
- پیاده سازی فصل 4-8 (پاداش)
- نمایش عمومی اطلاعات افراد تحت پیگیری شدید

3-4. مدیریت دادگاه و رأی نهایی (CourtViewSet)

قابلیت ها:

- ایجاد پرونده قضایی برای یک بازجویی تکمیل شده
- مشاهده پرونده های اختصاص داده شده به هر قاضی
- مشاهده جزئیات کامل پرونده (پرونده، مظنون، شواهد، بازجویی، افراد دخیل)
- ثبت رأی نهایی قاضی
- ثبت مجازات توسط قاضی یا رئیس پلیس

سطح دسترسی:

- فقط قاضی مجاز به ثبت رأی نهایی است.
- فقط قاضی یا رئیس پلیس مجاز به ثبت مجازات هستند.

پوشش نیازمندی پروژه:

- پیاده سازی کامل فصل 4-6 (محاكمه)
- ارائه گزارش جامع برای قاضی مطابق صورت پروژه

4. کنترل سطح دسترسی و امنیت

در این مازول موارد زیر رعایت شده است:

- استفاده از Permission های سفارشی مانند IsNotBaseUser و IsJudge
- اعمال فیلتر خودکار Queryset بر اساس نقش کاربر
- جلوگیری از عملیات غیرمجاز (مثلاً ثبت امتیاز توسط فرد غیرمرتبط با بازجویی)
- جلوگیری از ویرایش رأی دادگاه توسط افراد غیرمجاز

5. یکپارچگی با سایر ماژول‌ها

این ماژول به صورت مستقیم با بخش‌های زیر در تعامل است:

- ماژول Detection (دریافت مظنونین تأییدشده)
- ماژول Case (دریافت نوع جرم و سطح اهمیت آن)
- ماژول Evidences (نمایش شواهد برای قاضی)
- ماژول Users (کنترل نقش‌ها و سطوح دسترسی)

6. جمع‌بندی نهایی

ماژول Interrogation به صورت کامل فرآیند «از مظنون شدن تا صدور حکم نهایی» را مطابق صورت پروژه پیاده‌سازی کرده است. این ماژول:

- روند سلسله‌مراتبی تصمیم‌گیری پلیسی (کارآگاه → گروه‌بان → کاپیتان → رئیس پلیس) را پوشش می‌دهد.
- امکان ارجاع رسمی پرونده به دادگاه و صدور رأی نهایی را فراهم می‌کند.
- سیستم رتبه‌بندی مظنونین تحت پیگیری شدید و محاسبه پاداش را مطابق فرمول پروژه پیاده‌سازی می‌کند.
- با اعمال کنترل دسترسی دقیق و مدل‌سازی صحیح داده‌ها، از بروز تخلف منطقی در روندهای حساس جلوگیری می‌نماید.

گزارش فنی ماژول Rewards (مدیریت گزارش‌های مردمی و پرداخت پاداش)

1. هدف و دامنه ماژول

ماژول Rewards با هدف پیاده‌سازی سازوکار «گزارش اطلاعات مردمی مرتبط با پرونده‌ها و مظنونین» و تخصیص پاداش به گزارش‌های مؤثر طراحی شده است. این ماژول به شهروندان (Base User) امکان می‌دهد اطلاعات تکمیلی درباره پرونده‌های در جریان یا مظنونین تحت تعقیب را ثبت نمایند. سپس این اطلاعات طی یک فرآیند چندمرحله‌ای شامل بررسی اولیه توسط افسر و تأیید نهایی توسط کارآگاه مسئول پرونده ارزیابی می‌گردد. در صورت تأیید، پاداش مالی به گزارش‌دهنده تخصیص یافته و کد یکتای دریافت پاداش برای وی صادر می‌شود.

2. مدل داده و چرخه وضعیت‌ها (Data Model & State Machine)

2-1. مدل RewardTip

مدل RewardTip نمایانگر یک «اطلاعات/سرنخ ارسالی مردمی برای دریافت پاداش» است که به صورت مستقیم به یک پرونده (Case) یا یک مظنون (Suspect) مرتبط می‌شود.

ویژگی‌های اصلی مدل:

- شناسه یکتا (UUID) جهت افزایش امنیت و جلوگیری از حدس‌پذیری
- ثبت‌کننده اطلاعات (tip_submitter) از نوع کاربر عادی
- ارتباط اختیاری با:
 - یک پرونده مشخص
 - یا یک مظنون مشخص
- عنوان و توضیح تفصیلی اطلاعات
- زمان ثبت گزارش
- وضعیت فعلی بررسی
- اطلاعات مربوط به افسر بررسی‌کننده (افسر گشت یا پلیس)
- اطلاعات مربوط به کارآگاه تأییدکننده
- کد پاداش یکتا (reward_code)
- مبلغ پاداش تخصیص‌یافته
- زمان اطلاع‌رسانی به گزارش‌دهنده

چرخه وضعیت‌ها (RewardTipStatus):

- PENDING_REVIEW : در انتظار بررسی اولیه توسط افسر
- REJECTED : رد شده به دلیل بی‌اعتباری یا عدم انطباق
- SENT_TO_DETECTIVE : تأیید اولیه و ارسال به کارآگاه مسئول
- ACCEPTED : تأیید نهایی توسط کارآگاه و تخصیص پاداش
- CANCELLED : لغو شده (در صورت انصراف یا ابطال اداری)

قیود و منطق دامنه (Domain Logic):

- تولید کد پاداش تنها در صورت تأیید نهایی (ACCEPTED) انجام می‌شود.
- مبلغ پاداش فقط پس از تأیید کارآگاه قابل ثبت است.
- هر گزارش تنها یک بار وارد چرخه بررسی می‌شود و امکان بازگشت به مراحل قبلی وجود ندارد.

3. لایه Serializer و اعتبارسنجی داده‌ها

3-1. RewardTipCreateSerializer

- تنها فیلدهای مرتبط با ثبت گزارش توسط شهروند را دریافت می‌کند.

- اعتبارسنجی دامنه‌ای تضمین می‌کند که حداقل یکی از فیلدهای «پرونده مرتبط» یا «مظنون مرتبط» مقداری شده باشد.
- فیلد ثبت‌کننده به صورت خودکار از کاربر احراز هویت‌شده تنظیم می‌شود.

3-2. RewardTipListSerializer

- داده‌ها را به شکل مناسب برای نمایش در پنل کاربری یا داشبورد مدیریتی آماده می‌کند.
- نام کامل ثبت‌کننده، افسر بررسی‌کننده، کارآگاه تأییدکننده، عنوان پرونده و نام مظنون به صورت read-only و تجمیع‌شده بازگردانده می‌شود.

3-3. OfficerReviewSerializer و DetectiveConfirmSerializer

- تضمین می‌کنند که عملیات بررسی افسر و تأیید کارآگاه صرفاً با داده‌های معتبر انجام شود.
- حداقل مبلغ منطقی برای پاداش تعریف شده است تا از تخصیص مقادیر غیرواقعی جلوگیری شود.

4. API (ViewSet و Endpoints) ها و فرآیندهای اجرایی API

4-1. ثبت گزارش مردمی

Endpoint: POST /rewards/

دسترسی: فقط کاربران Base User
عملکرد:

- ثبت یک گزارش جدید توسط شهروند
- اتصال گزارش به پرونده یا مظنون
- قرار گرفتن خودکار در وضعیت «در انتظار بررسی اولیه افسر»

4-2. مشاهده گزارش‌های ثبت‌شده توسط کاربر

Endpoint: GET /rewards/my-tips/

دسترسی: کاربران Base User
عملکرد:

- بازبازی تمامی گزارش‌هایی که توسط کاربر جاری ثبت شده است
- افزایش شفافیت و امکان پیگیری وضعیت بررسی برای شهروندان

4-3. بررسی اولیه توسط افسر

Endpoint: POST /rewards/{id}/officer-review/

دسترسی: افسران پلیس و گشت
منطق کنترلی:

- فقط گزارش‌هایی با وضعیت PENDING_REVIEW قابل بررسی هستند.
- بررسی دسترسی از طریق منطق دامنه (can_be_reviewed_by) انجام می‌شود.

نتیجه عملیات:

- در صورت رد: وضعیت گزارش به REJECTED تغییر می‌کند.
- در صورت تأیید: گزارش به مرحله بررسی کارآگاه (SENT_TO_DETECTIVE) منتقل می‌شود.

4-4. تأیید نهایی توسط کارآگاه مسئول پرونده

Endpoint: POST /rewards/{id}/detective-confirm/

دسترسی: فقط کارآگاه مسئول پرونده/مظنون مرتبط
عملکرد:

- در صورت تأیید:
 - تغییر وضعیت به ACCEPTED
 - ثبت مبلغ پاداش
 - تولید کد یکتای دریافت پاداش
 - ثبت زمان اطلاع‌رسانی به گزارش‌دهنده
- در صورت رد:
 - تغییر وضعیت به REJECTED
 - ثبت یادداشت کارآگاه به عنوان مستند رد گزارش

4-5. لیست گزارش‌های در انتظار بررسی

Endpoints:

- GET /rewards/pending-for-officer/
- GET /rewards/pending-for-detective/

کاربرد:

- پشتیبانی از داشبورد کاری افسران و کارآگاهان
- کاهش خطای انسانی در تخصیص موارد به افراد غیرمسئول

4-6. استعلام وضعیت پاداش توسط شهروند

Endpoint: GET /rewards/lookup/?national_id=...&reward_code=...

عملکرد:

- شهروند با وارد کردن کد پاداش و شناسه هویتی خود می‌تواند صحت پاداش تخصیص یافته را استعلام نماید.
- تنها گزارش‌های تأیید شده (ACCEPTED) قابل استعلام هستند.
- در صورت عدم تطابق داده‌ها، پاسخ خطای معتبر بازگردانده می‌شود.

5. کنترل سطح دسترسی و امنیت

در این مازول کنترل‌های زیر اعمال شده است:

- جداسازی نقش‌ها:
 - شهروند (ثبت و پیگیری گزارش)
 - افسر (بررسی اولیه)
 - کارآگاه مسئول پرونده (تأیید نهایی)
- اعمال Permission های سفارشی (IsBaseUser, IsPoliceOfficer, IsDetective, IsPoliceNotCadet)
- جلوگیری از مشاهده یا بررسی گزارش‌ها توسط کاربران غیرمرتبط با پرونده
- استفاده از UUID برای جلوگیری از حدس زدن شناسه گزارش‌ها
- اعتبارسنجی هم در سطح Serializer و هم در منطق دامنه مدل

6. یکپارچگی با سایر مازول‌ها

ماژول Rewards به صورت مستقیم با بخش‌های زیر یکپارچه شده است:

- مازول Case (اتصال گزارش به پرونده‌های در جریان)
- مازول Interrogation (اتصال گزارش به مظنونین هر پرونده)
- مازول Detection (تشخیص کارآگاه مسئول هر پرونده از طریق رابطه Detection)
- مازول Users (کنترل نقش‌ها و گروه‌های کاربری)

7. پوشش نیازمندی‌های صورت پروژه

این مازول به طور مستقیم موارد زیر از نیازمندی‌های پروژه را پوشش می‌دهد:

- مشارکت مردمی در شناسایی مجرمین
- تخصیص پاداش برای گزارش‌های مؤثر
- امکان پیگیری وضعیت گزارش توسط شهروند
- کنترل دسترسی چندسطحی در فرآیند تصمیم‌گیری
- ثبت مستندات تصمیم افسر و کارآگاه جهت پاسخ‌گویی اداری

8. جمع‌بندی نهایی

ماژول Rewards با پیاده‌سازی یک فرآیند چندمرحله‌ای رسمی و قابل ردگیری برای بررسی گزارش‌های مردمی، ضمن افزایش مشارکت عمومی در حل پرونده‌ها، امکان تخصیص پاداش عادلانه و مستند را فراهم نموده است. طراحی مبتنی بر نقش، چرخه وضعیت شفاف، تولید کد یکتای پاداش و یکپارچگی با ماژول‌های پرونده، کشف و بازجویی موجب شده است این بخش از سامانه از منظر فنی و کسب‌وکاری نقش مکمل و ارزش‌افزوده‌ای در کل سیستم ایفا نماید.

گزارش فاز 2

گزارش فنی مؤلفه AddComplain (فرانت‌اند - ثبت و ویرایش شکایت)

1. هدف و جایگاه مؤلفه در معماری سامانه

مؤلفه AddComplain یکی از اجزای اصلی رابط کاربری سامانه مدیریت پرونده‌های جنایی است که وظیفه ثبت شکایت جدید و ویرایش شکایت‌های در حالت پیش‌نویس (Draft) یا بازگشتی را بر عهده دارد. این مؤلفه نقش درگاه ورودی اطلاعات شهروندان (شاکیان) به سامانه را ایفا می‌کند و ارتباط مستقیم میان کاربران نهایی و ماژول مدیریت پرونده‌ها در لایه بک‌اند برقرار می‌نماید.

طراحی تجربه کاربری این مؤلفه در راستای ایجاد حس مشارکت شهروندی در کشف جرم، از الگوهای تعاملی سیستم‌های کارآگاهی الهام گرفته شده است؛ از جمله سیستم‌های شناخته‌شده در آثار تعاملی نظیر L.A. Noire که بر ثبت دقیق وقایع، زمان و مکان حادثه و مشارکت چند شاهد تأکید دارند.

2. ورودی‌ها و قرارداد مؤلفه (Props & Interfaces)

این مؤلفه به صورت یک کامپوننت تابعی در چارچوب React پیاده‌سازی شده و ورودی‌های زیر را دریافت می‌کند:

- تابعی برای بستن دیالوگ ثبت شکایت : onClose
- onSave : تابعی برای ارسال داده نهایی شکایت به لایه بالاتر (State Management یا API)
- initialData : داده اولیه جهت ویرایش شکایت موجود
- currentUser : کاربر فعلی به عنوان شاکی اصلی
- availableUsers : فهرست کاربران قابل انتخاب به عنوان شاکیان اضافی

این طراحی، مؤلفه را از منطق ارسال به سرور مستقل کرده و امکان استفاده مجدد در سناریوهای مختلف (ایجاد یا ویرایش شکایت) را فراهم می‌سازد.

3. مدیریت وضعیت و چرخه حیات مؤلفه

3-1. مدیریت State

مؤلفه از Hook های استاندارد برای مدیریت وضعیت استفاده می‌کند:

- نگهداری فهرست شاکیان اضافی
- نگهداری نوع جرم انتخاب شده
- کنترل وضعیت فرم و مقداردهی اولیه هنگام ویرایش شکایت

3-2. مدیریت چرخه حیات (Lifecycle)

- هنگام بارگذاری مؤلفه، دیالوگ به صورت Modal نمایش داده می‌شود.
- رویداد فشردن کلید Escape و کلیک خارج از دیالوگ منجر به بسته شدن فرم می‌شود.
- این رفتار موجب بهبود تجربه کاربری و انطباق با الگوهای متداول رابط‌های مدرن شده است.

4. اعتبارسنجی داده‌ها در سمت کاربر (Client-side Validation)

پیش از ارسال داده‌ها به لایه بالاتر، اعتبارسنجی‌های زیر انجام می‌شود:

- الزام درج عنوان شکایت
- الزام درج شرح کامل حادثه
- الزام تعیین زمان وقوع
- الزام تعیین محل وقوع حادثه

در صورت نقص هر یک از فیلدهای الزامی، پیام هشدار مناسب به کاربر نمایش داده می‌شود. این رویکرد باعث کاهش خطاهای ورودی و کاهش بار پردازشی بک‌اند می‌گردد.

5. ساختار داده خروجی و آماده‌سازی برای ارسال به بک‌اند

پس از اعتبارسنجی موفق، داده‌ها در قالب ساختار RegisterComplain جمع می‌شوند. ویژگی‌های کلیدی این ساختار عبارت‌اند از:

- شناسه یکتا (در صورت ایجاد شکایت جدید)
- اطلاعات شاکی اصلی (کاربر جاری)
- عنوان، شرح، زمان و مکان وقوع حادثه
- نوع جرم (CrimeType)
- وضعیت اولیه شکایت (Draft)
- اطلاعات شاکیان اضافی (در صورت وجود)
- اطلاعات مربوط به تعداد دفعات بازبینی مجاز

این داده نهایی از طریق متد onSave به لایه والد ارسال می‌شود تا فرآیند ذخیره‌سازی یا ارسال به API انجام پذیرد.

6. طراحی تجربه کاربری (UI/UX)

6-1. فرم چندبخشی و ساختاریافته

فرم ثبت شکایت به بخش‌های منطقی زیر تقسیم شده است:

- اطلاعات پایه شکایت (عنوان و شرح)
- زمان و نوع جرم
- محل وقوع
- مدیریت شاکیان اضافی
- نمایش وضعیت شکایت (در صورت ویرایش شکایت موجود)

6-2. نمایش بصری شدت جرم

با انتخاب نوع جرم، یک نشانگر رنگی (سبز، زرد، نارنجی، قرمز) سطح اهمیت جرم را نمایش می‌دهد. این نمایش بصری به کاربر کمک می‌کند درک بهتری از طبقه‌بندی جرم داشته باشد و داده‌ها را با دقت بیشتری وارد کند.

6-3. پشتیبانی از چند شاکی

امکان افزودن چند شاکی اضافی در رابط کاربری پیش‌بینی شده است. کاربر می‌تواند افراد مرتبط با حادثه را از میان کاربران موجود انتخاب کرده و رابطه آن‌ها با حادثه را مشخص نماید. این قابلیت به پوشش سناریوهای واقعی‌تر (پرونده‌های دارای چند شاکی یا شاهد) کمک می‌کند.

7. ملاحظات دسترس‌پذیری (Accessibility)

- استفاده از `aria-modal="true"` و `role="dialog"` برای سازگاری با ابزارهای کمکی
 - تعیین برجسب‌های مناسب برای فیلدهای ورودی
 - پشتیبانی از ناوبری با صفحه‌کلید (بستن دیالوگ با Escape)
 - تمرکز خودکار روی فیلد عنوان در شروع کار
-

8. ارتباط با نیازمندی‌های پروژه

این مؤلفه نیازمندی‌های زیر را در پروژه پوشش می‌دهد:

- امکان ثبت شکایت توسط شهروند
 - امکان ویرایش شکایت‌های پیش‌نویس یا بازگشتی
 - ثبت اطلاعات دقیق حادثه شامل زمان، مکان و نوع جرم
 - پشتیبانی از چند شکایت در یک پرونده
 - آماده‌سازی داده‌ها برای ارسال به بک‌اند مطابق قرارداد API
-

9. جمع‌بندی نهایی

مؤلفه `AddComplain` به عنوان درگاه اصلی ورود داده‌های شکایت در لایه فرانت‌اند، نقش کلیدی در یکپارچگی تجربه کاربری سامانه ایفا می‌کند. این مؤلفه با بهره‌گیری از مدیریت وضعیت مناسب، اعتبارسنجی سمت کاربر، طراحی رابط کاربری ساختاریافته و رعایت اصول دسترس‌پذیری، امکان ثبت دقیق و قابل اتکای شکایات را فراهم ساخته و بستر لازم برای پردازش صحیح داده‌ها در لایه بک‌اند را مهیا می‌نماید.

این بخش از پروژه از منظر «تعامل کاربر نهایی با سیستم ثبت پرونده» به صورت کامل پیاده‌سازی شده و نیازمندی‌های مربوط به ثبت شکایت در سامانه را پوشش داده است.

گزارش فنی مؤلفه BoardList (فرانت‌اند - نمایش فهرست پرونده‌ها)

1. هدف و جایگاه مؤلفه در معماری سامانه

مؤلفه `BoardList` وظیفه نمایش فهرست پرونده‌ها/شکایت‌ها (Cases) را در رابط کاربری سامانه مدیریت پرونده‌های جنایی بر عهده دارد. این مؤلفه به کاربران اجازه می‌دهد وضعیت پرونده‌ها را مشاهده کنند، پرونده مورد نظر را انتخاب کرده و در صورت مجاز بودن، پرونده جدید ایجاد نمایند.

مؤلفه BoardList به صورت کامپوننت تابعی در React پیاده سازی شده و کاملاً مستقل از لایه داده (Data Layer) طراحی شده است؛ داده ها از والد (Parent Component) به صورت props دریافت می شوند و تعاملات (انتخاب یا ایجاد پرونده) نیز از طریق callback به والد اطلاع داده می شوند.

2. ورودی ها و قرارداد مؤلفه (Props)

مؤلفه ورودی های زیر را دریافت می کند:

- boards (Array): عنوان، وضعیت، مسئول، شامل شناسه، پرونده و تعداد لیدها
- onBoardSelect (Function): برای انتخاب یک پرونده callback تابع: بالاتر
- canCreate (Boolean): مشخص می کند آیا کاربر اجازه ایجاد پرونده جدید دارد یا خیر
- onCreateNew (Function): برای ایجاد پرونده جدید در صورت فعال بودن قابلیت callback تابع: ایجاد

این طراحی امکان استفاده مجدد مؤلفه در بخش های مختلف سامانه (مانند داشبورد کاربری یا صفحه مدیریت پرونده ها) را فراهم می کند.

3. مدیریت داده ها و محاسبات داخلی

3-1. آمار پرونده ها

مؤلفه به صورت خودکار آمار کلی پرونده ها را محاسبه می کند:

- total: تعداد کل پرونده ها
- active: (status === "active") تعداد پرونده های فعال
- pending: (status === "pending_review") تعداد پرونده های در انتظار بازبینی
- closed: (status === "closed") تعداد پرونده های بسته شده

این آمار در بالای صفحه به کاربران نمایش داده می شود و امکان مشاهده سریع وضعیت کلی پرونده ها را فراهم می کند.

4. طراحی رابط کاربری (UI/UX)

4-1. هدر و آمار

- نمایش عنوان بخش (Active Cases)
- نمایش آمار کل، فعال و در انتظار بازبینی با فونت و رنگ برجسته
- دکمه ایجاد پرونده جدید (+ New Case) در صورت فعال بودن قابلیت ایجاد پرونده

4-2. نمایش کارتهای پروندهها

- هر پرونده به صورت کارت نمایش داده می شود و شامل موارد زیر است:
 - شناسه پرونده: `#XXXXXXXX` Case (۸ کاراکتر اول UUID)
 - وضعیت: Active, Pending Review, Closed با رنگ بندی ویژه
 - عنوان پرونده: `board.case_title` یا مقدار پیش فرض `Untitled Case`
 - جزئیات متا: تعداد لیدها و نام سرچنت مسئول پرونده
 - نوار پیشرفت: درصد تکمیل پرونده (`board.progress`)
- کارتها به صورت **Grid Responsive** نمایش داده می شوند تا در اندازه های مختلف صفحه نمایش بهینه باشند.

4-3. حالت بدون پرونده (Empty State)

- در صورتی که کاربر پرونده ای نداشته باشد، پیغام مناسب (No cases assigned) نمایش داده می شود.
- در صورت امکان ایجاد پرونده، دکمه ایجاد پرونده جدید به کاربر پیشنهاد می شود (Create your first case).

این طراحی تجربه کاربری مناسبی ایجاد می کند و کاربران را به اقدام بعدی هدایت می کند.

5. استایل و دسترس پذیری

- استفاده از **CSS-in-JS** برای تعریف تمامی استایلها به صورت شیء جاوااسکریپت (`styles`)
- کارتها دارای **Shadow, Border Radius** و **Hover Effect** برای جذابیت بصری و تشخیص تعاملی بودن هستند
- رنگها برای وضعیت پروندهها انتخاب شده اند تا کاربران به سرعت وضعیت پرونده را تشخیص دهند:
 - Active: سبز روشن (`#d4edda`)
 - Pending Review: زرد (`#fff3cd`)
 - Closed: قرمز (`#f8d7da`)
- عناصر متنی با کنتراست مناسب و فونت خوانا طراحی شده اند
- دکمه ها دارای حالت `hover` و `cursor pointer` برای نمایش تعامل پذیری

6. تعامل با کاربران

- انتخاب پرونده: کلیک روی هر کارت پرونده باعث فراخوانی `onBoardSelect(board)` می‌شود.
- ایجاد پرونده جدید: در صورت فعال بودن قابلیت `canCreate`، دکمه ایجاد پرونده فعال است و با کلیک، `onCreateNew()` فراخوانی می‌شود.

این طراحی ساده و مستقیم، امکان مدیریت پرونده‌ها و اقدامات فوری را برای کاربر فراهم می‌کند.

7. ارتباط با نیازمندی‌های پروژه

مؤلفه `BoardList` بخش قابل توجهی از نیازمندی‌های پروژه را پوشش می‌دهد:

- نمایش تمام پرونده‌های کاربر و وضعیت آن‌ها
 - امکان انتخاب یک پرونده برای مشاهده جزئیات
 - ارائه اطلاعات متا شامل لیدها و مسئول پرونده
 - نمایش پیشرفت پرونده برای آگاهی سریع از وضعیت پرونده‌ها
 - ایجاد پرونده جدید در صورت مجاز بودن کاربر
-

8. جمع‌بندی نهایی

مؤلفه `BoardList` به عنوان بخشی کلیدی از داشبورد سامانه، تجربه کاربری واضح و مدیریت‌شده‌ای برای کاربران فراهم می‌کند.

ویژگی‌های اصلی:

- نمایش آماری: وضعیت کل پرونده‌ها و درصد پیشرفت آن‌ها
- نمایش کارت‌های پرونده: طراحی قابل فهم و واکنش‌گرا
- حالت بدون پرونده: ارائه راهنمایی مناسب به کاربر
- پشتیبانی از تعاملات: انتخاب و ایجاد پرونده‌ها با `callback`

این مؤلفه، نیازمندی‌های فرانت‌اند برای مدیریت و نمایش پرونده‌ها را به صورت کامل و بهینه پیاده‌سازی کرده است.

گزارش فنی مؤلفه `ComplaintList` (فرانت‌اند - مدیریت شکایات)

1. هدف و جایگاه مؤلفه در معماری سامانه

مؤلفه `ComplaintList` در سامانه مدیریت پرونده‌ها و شکایات وظیفه دارد لیستی از شکایات ثبت‌شده را به کاربران نمایش دهد، امکان ایجاد شکایت جدید و ویرایش شکایات موجود را فراهم کند و جزئیات مهم هر شکایت را به شکل واضح ارائه دهد.

این مؤلفه به صورت کامپوننت تابعی در `React` با `TypeScript` پیاده‌سازی شده و داده‌ها از طریق `state` داخلی یا `props` مدیریت می‌شوند. این طراحی، انعطاف‌پذیری برای تعامل با `API` و سایر کامپوننت‌ها را فراهم می‌کند.

2. ورودی‌ها و داده‌های داخلی (Props & State)

2-1. State داخلی

- `isAddModalOpen` (Boolean): مدیریت باز یا بسته بودن مودال ثبت/ویرایش شکایت.
- `complaints` (Array): آرایه‌ای از شکایات ثبت‌شده.
- `editingComplaint` (RegisterComplain | undefined): شکایتی که در حال ویرایش است.

2-2. داده‌های نمونه و کاربر

- `currentUser`: برای نمایش و ثبت شکایات (Mock) کاربر فعلی.
- `availableUsers`: آرایه‌ای از کاربران موجود برای اضافه شدن به شکایت به عنوان شاکی.
- `mockComplaint`: شکایت نمونه برای تست و نمایش قابلیت ویرایش.

3. عملکرد و منطق داخلی مؤلفه

3-1. مدیریت افزودن و ویرایش شکایت

- افزودن شکایت جدید: تابع `handleSaveComplaint` شکایت جدید را به ابتدای آرایه `complaints` اضافه می‌کند.
- ویرایش شکایت موجود: همان تابع، اگر `editingComplaint` وجود داشته باشد، شکایت را به‌روز رسانی می‌کند.
- پس از ذخیره، مودال بسته شده و وضعیت ویرایش بازنشانی می‌شود.

3-2. باز و بسته کردن مودال

- برای بستن مودال و پاک کردن وضعیت ویرایش `handleCloseModal`.
- باز کردن مودال برای شکایت جدید یا ویرایش با تعیین `editingComplaint` انجام می‌شود.

3-3. نمایش لیست شکایات

- در صورت وجود شکایت‌ها، هر شکایت در کارت جداگانه نمایش داده می‌شود.
- هر کارت شامل:
 - عنوان شکایت (complaint.title)
 - توضیحات کوتاه (complaint.description) با محدودیت دو خط (line-clamp-2)
 - زمان و مکان وقوع
 - نوع جرم (crime_type)
 - دکمه ویرایش جهت باز کردن مودال با داده‌های موجود
- وضعیت شکایت با Badge رنگی برای نمایش وضعیت: Draft, Pending, Approved, Rejected

3-4. حالت بدون شکایت (Empty State)

- در صورت خالی بودن آرایه complaints :
- پیام مناسب به کاربر نمایش داده می‌شود.
- راهنمایی برای ثبت اولین شکایت با دکمه "ثبت شکایت جدید".
- طراحی به صورت کارت متمرکز با آیکون مناسب و رنگ‌بندی برای جذابیت بصری.

4. طراحی رابط کاربری (UI/UX)

4-1. هدر و دکمه‌ها

- عنوان صفحه: مدیریت شکایات
- دکمه افزودن شکایت جدید با رنگ آبی، آیکون + و افکت hover
- دکمه ویرایش نمونه برای نمایش و تست قابلیت ویرایش

4-2. کارت شکایات

- رنگ پس‌زمینه سفید در حالت روشن و خاکستری تیره در حالت تاریک (Dark Mode)
- برای برجسته شدن کارت‌ها Shadow و Border Radius
- اطلاعات شکایت به صورت سلسله‌مراتبی و خوانا

4-3. Badge وضعیت

- رنگ‌بندی متناسب با وضعیت شکایت:
- Draft → خاکستری
- Pending → زرد
- Approved → سبز
- Rejected → قرمز
- متن وضعیت در داخل Badge نمایش داده می‌شود تا کاربر به سرعت از وضعیت شکایت آگاه شود.

5. تعامل با کاربران

- افزودن شکایت جدید: کلیک روی دکمه "ثبت شکایت جدید"
- ویرایش شکایت موجود: کلیک روی دکمه ویرایش داخل هر کارت یا ویرایش نمونه
- باز و بسته کردن مودال: از طریق State داخلی کنترل می‌شود

این طراحی امکان مدیریت سریع شکایات را فراهم می‌کند و تجربه کاربری روانی ایجاد می‌کند.

6. اتصال به API و جریان داده

6-1. ارسال شکایت به API

- در کامپوننت `ComplaintCreationFlow` تابع `handleSubmitToApi` :
- داده‌ها برای API آماده می‌شوند (شامل عنوان، توضیحات، زمان و مکان، نوع جرم و شاکیان)
- ارسال POST به مسیر `/api/complaints/`
- مدیریت خطا و نمایش پیام موفقیت یا خطا به کاربر
- این بخش امکان تکمیل فرآیند ثبت شکایت در Backend را فراهم می‌کند.

6-2. جزئیات شکایت

- در `ComplaintDetailPage` :
 - دریافت شناسه شکایت از URL با `( useParams )` React Router
 - نمایش جزئیات شکایت
 - امکان ورود به حالت ویرایش و باز کردن مودال با داده‌های موجود
-

7. استفاده از مؤلفه‌های فرعی

- `AddComplain` :
 - مودال ثبت یا ویرایش شکایت
 - دریافت Props شامل `onSave` , `onClose` , داده اولیه، کاربر جاری و کاربران موجود
 - این کامپوننت تعامل مستقیم کاربر با فرم را مدیریت می‌کند و وضعیت شکایت‌ها را در `parent` به‌روزرسانی می‌کند
-

8. دسترسی و قابلیت‌های اضافی

- واکنش‌گرا (Responsive) با استفاده از Tailwind CSS
- پشتیبانی از Dark Mode با رنگ‌بندی متناسب
- استفاده از افکت‌های Transition و Hover برای جذابیت بصری
- فرمت تاریخ به فارسی با toLocaleDateString("fa-IR")
- مدیریت ویرایش و ایجاد شکایت با یک Flow یکتا و ساده

9. ارتباط با نیازمندی‌های پروژه

مؤلفه ComplaintList بخش بزرگی از نیازمندی‌های سامانه را پوشش می‌دهد:

1. مدیریت شکایات: نمایش، ایجاد و ویرایش شکایات
2. جزئیات شکایت: زمان، مکان، نوع جرم، شاکیان و وضعیت
3. حالت خالی: راهنمایی کاربر برای ثبت اولین شکایت
4. ارتباط با Backend: قابلیت ارسال و دریافت داده‌ها با API
5. تجربه کاربری مناسب: کارت‌ها، Badge وضعیت، مودال‌های واضح و قابل تعامل

10. جمع‌بندی

مؤلفه ComplaintList و کامپوننت‌های مرتبط آن (AddComplain، ComplaintDetailPage) یک جریان کامل مدیریت شکایات در سامانه را فراهم می‌کنند که شامل:

- ثبت و ویرایش شکایت
- نمایش جزئیات و وضعیت‌ها
- واکنش‌گرایی و پشتیبانی از حالت تاریک
- ارتباط امن با Backend

این مؤلفه نیازمندی‌های فرانت‌اند برای مدیریت شکایات را به صورت کامل و بهینه پیاده‌سازی کرده است.

گزارش فنی مؤلفه Dashboard (فرانت‌اند – داشبورد کاربری)

1. هدف و جایگاه مؤلفه در معماری سامانه

مؤلفه Dashboard در سامانه پلیس و مدیریت پرونده‌ها، به عنوان صفحه اصلی کاربر پس از ورود عمل می‌کند. وظایف اصلی این صفحه عبارتند از:

1. احراز هویت و مدیریت دسترسی کاربران: بررسی وضعیت ورود و گروه کاربری

2. تعیین دسترسی‌ها و مجوزهای کاربر: ارائه قابلیت‌های متناسب با نقش کاربری
 3. نمایش مازول‌های سامانه: بارگذاری و نمایش مؤلفه `ModulesRegistry` که شامل دسترسی به بخش‌های مختلف سامانه است
 4. امکان خروج از سامانه (`Logout`)
- این مؤلفه یک کامپوننت تابعی `React` است و با `Context` احراز هویت (`AuthContext`) تعامل دارد.

2. ورودی‌ها و داده‌های داخلی (Props & State)

2-1. داده‌های Context

- شیء کاربر جاری شامل: `user`
 - شناسه و نام کاربری
 - نام کامل (`full_name`)
 - گروه‌ها (`groups`) که نقش کاربر را مشخص می‌کند
- تابع خروج کاربر: `logout`
- نشان‌دهنده وضعیت ورود کاربر `Boolean`: `isAuthenticated`

2-2. داده‌های داخلی

- آرایه‌ای از مجوزهای کاربر که بر اساس گروه‌های کاربری تعیین می‌شود: `userPermissions`
- نکته: مؤلفه به صورت مستقیم `State` داخلی `React` ندارد و تمام منطق از `Context` و محاسبه در زمان `Render` مدیریت می‌شود.

3. عملکرد و منطق داخلی مؤلفه

3-1. مدیریت دسترسی و احراز هویت

- بررسی وضعیت ورود با `isAuthenticated`:

```
    } if (!isAuthenticated)
      ;navigate("/login")
      ;return null
    {
```

- اگر کاربر وارد نشده باشد، به صفحه `login/` هدایت می‌شود.

3-2. تعیین مجوزهای کاربر

- آرایه `userPermissions` بر اساس گروه‌های کاربری ایجاد می‌شود:
- کاربر گروه **Detective**:
- مجوزهای کامل: مشاهده، ایجاد و ویرایش پرونده‌ها، ایجاد سرخ، پیشنهاد مزنون
- سایر کاربران:
- فقط مجوز مشاهده پرونده‌ها (`cases.view`)
- مجوزها به `ModulesRegistry` پاس داده می‌شوند تا نمایش مژول‌ها بر اساس نقش کاربر صورت گیرد.

3-3. تعامل با Context

- خروج از سامانه (Logout) با دکمه Logout فعال می‌شود:

```
<button onClick={logout}>Logout</button>
```

4. طراحی رابط کاربری (UI/UX)

4-1. هدر

- عنوان صفحه: L.A. Noire Dashboard - Detective > نام کاربر
- دکمه خروج: رنگ قرمز با افکت `hover`، دسترسی واضح و سریع

4-2. محتوای اصلی

- کارت سفید با `Shadow` و `Border Radius` برای نمایش محتوای داشبورد
- حداقل ارتفاع: 500px برای حفظ ظاهر منظم و جایگذاری مژول‌ها

4-3. پاسخگویی و مرکزیت

- حداکثر عرض کانتینر: 1200px
- مرکز چین شده و با `padding` مناسب برای فاصله از لبه‌ها

5. تعامل با مؤلفه‌های فرعی

- `ModulesRegistry`:
- دریافت `props`:
- کاربر جاری: `user`
- مجوزهای کاربر: `permissions`
- این مؤلفه مسئول نمایش مژول‌ها و بخش‌های سامانه بر اساس دسترسی کاربر است

6. قابلیت‌ها و ویژگی‌های کلیدی

1. مدیریت دسترسی پویا:
 - بررسی گروه‌های کاربری و اختصاص مجوزهای متفاوت
2. حفاظت از صفحات خصوصی:
 - هدایت خودکار به صفحه Login در صورت عدم احراز هویت
3. سازگاری با Context:
 - استفاده از AuthContext برای وضعیت ورود و اطلاعات کاربر
4. نمایش کاربر:
 - نام کاربر یا نام کامل در هدر برای شخصی‌سازی تجربه کاربری
5. قابلیت Logout:
 - امکان خروج سریع با یک دکمه مشخص

7. ارتباط با نیازمندی‌های پروژه

مؤلفه Dashboard نیازمندی‌های زیر را پوشش می‌دهد:

1. ورود و احراز هویت کاربران: کنترل دسترسی و هدایت به صفحه Login در صورت عدم ورود
2. مدیریت مجوزها بر اساس نقش کاربر: Detective یا سایر کاربران
3. نمایش ماژول‌های سیستم: اتصال به ModulesRegistry
4. تجربه کاربری ساده و واضح: هدر با نام کاربر و دکمه Logout
5. امنیت پایه فرانت‌اند: جلوگیری از دسترسی مستقیم به صفحات بدون احراز هویت

8. جمع‌بندی

مؤلفه Dashboard یک صفحه مرکزی و امن برای کاربران سامانه است که:

- با Context احراز هویت یکپارچه شده
- دسترسی‌ها و مجوزهای کاربر را به صورت پویا مدیریت می‌کند
- ماژول‌های سامانه را بر اساس نقش کاربر نمایش می‌دهد
- امکان خروج سریع و مدیریت امن جلسه کاربری را فراهم می‌کند

این طراحی، پایه‌ای برای یک داشبورد کاربری حرفه‌ای و قابل گسترش در سامانه‌های مدیریت پرونده و شکایات محسوب می‌شود.

گزارش فنی مؤلفه DetectiveBoard (فرانت‌اند - برد کارآگاه)

1. هدف و جایگاه مؤلفه در سامانه

مؤلفه DetectiveBoard یک برد تعاملی برای مدیریت سرخ‌ها و ارتباطات پرونده‌ها در سامانه مدیریت پرونده و تحقیقات پلیس است. وظایف اصلی این مؤلفه عبارتند از:

1. نمایش لیست سرخ‌ها (Leads) با جزئیات مرتبط با هر سرخ
2. ایجاد و ویرایش سرخ‌ها با قابلیت تعیین نوع (Evidence یا Note)
3. نمایش و مدیریت اتصالات بین سرخ‌ها (Yarns)
4. پیشنهاد مظنونان (Suspect Suggestions)
5. مدیریت دسترسی‌ها بر اساس مجوزهای کاربر (ویرایش پرونده، ایجاد سرخ، پیشنهاد مظنون)
6. تعامل با Modals برای ایجاد/ویرایش/حذف سرخ و مدیریت اتصالات

این مؤلفه یک کامپوننت تابعی React است و به صورت مستقیم با API سامانه تعامل دارد.

2. ورودی‌ها و Props

2-1. Props مؤلفه

- boardId : شناسه برد پرونده
- boardData : داده‌های کلی برد (برای اطلاعات اضافی و رفرش)
- onBoardUpdate : تابع برای به‌روزرسانی داده‌های والد پس از تغییرات
- permissions : آرایه‌ای از مجوزهای کاربر برای تعیین قابلیت‌های فعال

2-2. State داخلی

- leads : آرایه سرخ‌های برد
- yarns : آرایه اتصالات بین سرخ‌ها
- suggestions : آرایه پیشنهادهای مظنون
- loading , error : وضعیت لودینگ و خطاها
- showLeadModal , showYarnModal , showSuggestionModal : وضعیت نمایش مودال‌ها
- selectedLead : سرخ انتخاب شده برای ویرایش
- connectingMode : حالت اتصال سرخ‌ها
- selectedLeadForConnection : سرخ انتخاب شده برای شروع اتصال

3. مدیریت دسترسی و مجوزها

- تعیین قابلیت‌های کاربر بر اساس آرایه permissions :
 - `cases.edit` → ویرایش برد و سرخ‌ها
 - `leads.create` → ایجاد سرخ جدید
 - `suspects.suggest` → پیشنهاد مظنون
 - نمایش یا فعال‌سازی دکمه‌ها و تعاملات برد بر اساس این مجوزها
- این رویکرد باعث می‌شود فقط کاربران مجاز به تغییر برد و سرخ‌ها باشند و کاربران دیگر صرفاً قادر به مشاهده اطلاعات باشند.

4. عملیات CRUD و تعامل با API

4-1. مدیریت سرخ‌ها (Leads)

- ایجاد سرخ: `handleCreateLead`
- ارسال درخواست `POST /api/detection/leads` با داده‌های سرخ و `boardId`
- به‌روزرسانی لیست سرخ‌ها پس از موفقیت
- ویرایش سرخ: `handleUpdateLead`
- ارسال درخواست `PUT /api/detection/leads/:id`
- به‌روزرسانی لیست سرخ‌ها پس از موفقیت
- حذف سرخ: `handleDeleteLead`
- ارسال درخواست `DELETE /api/detection/leads/:id`
- رفرش داده‌ها پس از حذف

4-2. مدیریت اتصالات (Yarns)

- ایجاد اتصال بین دو سرخ: `handleCreateYarn`
- ارسال درخواست `POST /api/detection/yarns` با شناسه سرخ‌ها
- پشتیبانی از حالت `connectingMode` برای انتخاب دوتایی سرخ‌ها
- حذف اتصال: `handleDeleteYarn`
- ارسال درخواست `DELETE /api/detection/yarns/:id`

4-3. مدیریت پیشنهاد مظنون

- نمایش مودال `SuspectSuggestionModal` و رفرش داده‌ها پس از ثبت پیشنهاد

5. نمایش داده‌ها و رابط کاربری

5-1. Toolbar

- دکمه‌ها نمایش داده می‌شوند بر اساس مجوزها:
- **+ New Lead** → ایجاد سرنخ
- **Connect Leads** → فعال‌سازی حالت اتصال سرنخ‌ها (در صورتی که حداقل دو سرنخ موجود باشد)
- **Suggest Suspects** → پیشنهاد مظنون

5-2. نمایش سرنخ‌ها

- هر سرنخ به صورت یک کارت قابل جابجایی نمایش داده می‌شود:
- نوع سرنخ: **Evidence (E)** یا **Note (N)**
- مختصات برد با درصد برای قرارگیری روی داشبورد
- اطلاعات Evidence شامل نوع و توضیحات (در صورت موجود بودن)
- امکان کلیک راست برای ویرایش در صورت داشتن مجوز

5-3. نمایش Yarn‌ها

- خطوط SVG بین سرنخ‌ها برای نمایش ارتباطات
- نقاط میانی برای حذف اتصال در صورت داشتن مجوز
- طراحی ریسپانسیو با مقیاس‌بندی درصدی

5-4. Modals

- **LeadModal** → ایجاد/ویرایش/حذف سرنخ
- **YarnModal** → مدیریت اتصال‌ها
- **SuspectSuggestionModal** → ثبت پیشنهاد مظنون

6. جریان داده‌ها

1. لود داده‌ها با `fetchBoardData` هنگام `mount` مؤلفه
 2. نمایش سرنخ‌ها و اتصال‌ها بر اساس داده‌های API
 3. تعامل کاربر:
 - ایجاد، ویرایش، حذف سرنخ
 - ایجاد و حذف Yarn
 - پیشنهاد مظنون
 4. به‌روزرسانی خودکار برد بعد از هر تغییر موفق
-

7. ویژگی‌های کلیدی و نقاط قوت

1. مدیریت پیشرفته سرخ‌ها و اتصالات برای تحقیقات پرونده
 2. مدیریت دسترسی پویا با مجوزهای دقیق
 3. تعامل گرافیکی و ریسپانسیو با مکان‌یابی سرخ‌ها و نمایش ارتباط‌ها
 4. پشتیبانی از چندین مودال برای ایجاد، ویرایش و پیشنهاد مظنون
 5. استفاده از API REST برای همگام‌سازی داده‌ها با سرور
-

8. ارتباط با نیازمندی‌های پروژه

- ایجاد و مدیریت سرخ‌ها و اتصالات → نیازمندی اصلی برد کارآگاه
 - مدیریت مجوزها بر اساس نقش کاربر → تضمین امنیت و کنترل دسترسی
 - پشتیبانی از پیشنهاد مظنون → تعامل با بخش تحلیل جرم
 - رابط گرافیکی تعاملی → بهبود تجربه کاربری و کارایی تحقیقات
-

9. جمع‌بندی

مؤلفه DetectiveBoard یک برد کارآگاه دیجیتال پویا برای مدیریت سرخ‌ها، اتصالات بین آن‌ها و پیشنهاد مظنون ارائه می‌دهد. این مؤلفه به صورت کاملاً مجوزمحور طراحی شده و تعاملات گرافیکی و CRUD سرخ‌ها و Yarn‌ها را به طور کامل مدیریت می‌کند. این بخش، هسته قابلیت‌های تحقیقات دیجیتال در سامانه محسوب می‌شود و امکان توسعه و گسترش آینده را نیز فراهم می‌کند.

گزارش فنی مؤلفه Home (فرانت‌اند - صفحه اصلی)

1. هدف و جایگاه مؤلفه در سامانه

مؤلفه Home صفحه اصلی سامانه مدیریت پرونده‌های جنایی است که به کاربران کمک می‌کند:

1. معرفی سامانه و اهداف آن
2. نمایش آمار کلی سامانه شامل پرونده‌ها، کارمندان و پرونده‌های فعال
3. ارائه دکمه‌های دسترسی سریع برای ورود و ثبت‌نام کاربران
4. ایجاد تجربه اولیه کاربری جذاب و بصری با طراحی مدرن و ریسپانسیو

این صفحه نقش صفحه لندینگ سامانه را دارد و کاربر را برای ورود به سامانه یا ثبت‌نام هدایت می‌کند.

2. ساختار مؤلفه

2-1. Hooks و State

- `useAuth()` → استخراج اطلاعات کاربر وارد شده
- `useState` برای مدیریت:
 - `stats` → آمار سامانه:
 - `total_solved_cases` → تعداد پرونده‌های حل شده
 - `total_employees` → تعداد کارمندان فعال
 - `active_cases` → تعداد پرونده‌های فعال
 - `loading` → وضعیت لودینگ هنگام دریافت آمار
- `useEffect` → فراخوانی داده‌های آماری سامانه از API:

```
fetch('/api/case/stats/')
```

و بروزرسانی `stats` state پس از دریافت داده‌ها.

2-2. Props

صفحه **Home** فاقد props ورودی است و به صورت مستقیم با context احراز هویت و API تعامل دارد.

3. طراحی رابط کاربری

3-1. Header (هدر اصلی)

- پس‌زمینه با گرادیانت چند رنگ برای جلوه بصری
- لایه نیمه شفاف (`backdrop-blur`) برای افزایش کنتراست متن
- شامل:
 - **عنوان سامانه:** "سامانه مدیریت پرونده‌های جنایی"
 - توضیح کوتاه درباره سامانه
 - دکمه‌های دسترسی سریع:
 - ورود به سامانه (`login/`)
 - ثبت‌نام (`register/`)
- طراحی ریسپانسیو با کلاس‌های Tailwind CSS (`flex-col sm:flex-row`)

3-2. About Section (درباره سامانه)

- توضیح کوتاه درباره هدف سامانه
- طراحی با:

- پس‌زمینه شفاف و blur
- گوشه‌های گرد و سایه
- متن مرکزی و قابل خواندن

3-3. Stats Section (آمار سامانه)

- نمایش سه آمار اصلی:
 1. پرونده‌های حل شده (total_solved_cases)
 2. کارمندان فعال (total_employees)
 3. پرونده‌های فعال (active_cases)
- هر کارت دارای:
 - عنوان آمار
 - مقدار عددی با قالب‌بندی toLocaleString()
 - رنگ متناسب برای هر کارت
 - انیمیشن hover و سایه برای جذابیت
- استفاده از Tailwind grid برای ریسپانسیو بودن کارت‌ها

3-4. Call-to-Action Section (CTA – دعوت به ورود)

- پس‌زمینه گرادیانت مشابه هدر
- پیام جذاب برای ترغیب کاربر به ورود
- دکمه بزرگ ورود به سامانه با انیمیشن hover و scale

4. تعامل با API

1. Endpoint استفاده شده: /api/case/stats/

2. روش درخواست: GET

3. داده دریافت شده: JSON شامل:

```

    }
    ,total_solved_cases": 123"
    ,total_employees": 45"
    active_cases": 67"
  {

```

4. داده‌ها پس از دریافت در state stats ذخیره می‌شوند

5. در هنگام لودینگ، مقدار آمار به صورت ... نمایش داده می‌شود

خطای دریافت داده‌ها با console.error ثبت می‌شود و تجربه کاربری به هم نمی‌ریزد.

5. طراحی ریسپانسیو و حالت تاریک (Dark Mode)

• استفاده از کلاس‌های Tailwind CSS:

- `dark:bg-gray-950` , `dark:text-gray-300`
 - برای طراحی واکنش‌گرا `flex-col` و `flex-row` `sm:`
 - با قابلیت سازگاری با حالت تاریک `blur` و افکت `Gradient`
-

6. نکات فنی

1. تعامل ساده و مستقیم با `context` احراز هویت برای دسترسی به کاربر جاری
 2. استفاده از `Link` از `react-router-dom` برای هدایت به صفحات ورود و ثبت‌نام بدون `reload`
 3. مدیریت وضعیت `لودینگ` هنگام دریافت آمار برای جلوگیری از نمایش خالی یا خطا
 4. طراحی گرافیکی با ترکیب `blur` , `gradient` و `shadow` برای ظاهر مدرن و جذاب
-

7. ارتباط با نیازمندی‌های پروژه

- معرفی سامانه به کاربر جدید → الزامی برای UX اولیه
 - امکان دسترسی سریع به ورود و ثبت‌نام → مسیر اصلی تعامل با سامانه
 - نمایش آمار واقعی سامانه → افزایش اعتماد کاربر و شفافیت اطلاعات
 - طراحی واکنش‌گرا و حالت تاریک → افزایش قابلیت استفاده و تطابق با استانداردهای مدرن
-

8. جمع‌بندی

مؤلفه `Home` یک صفحه `لندینگ` کامل و مدرن برای سامانه مدیریت پرونده‌های جنایی فراهم می‌کند. این صفحه ترکیبی از:

- معرفی سامانه و اهداف آن
- نمایش آمار کلیدی سامانه
- طراحی ریسپانسیو و حالت تاریک
- و دکمه‌های ورود و ثبت‌نام `CTA`

است و به عنوان نقطه شروع تعامل کاربران با سامانه عمل می‌کند.

گزارش فنی مؤلفه LeadModal (فرانت‌اند - مودال مدیریت Lead)

1. هدف و جایگاه مؤلفه در سامانه

مؤلفه LeadModal یک پنجره مودال (Popup) برای ایجاد، ویرایش و حذف اطلاعات یک Lead در سامانه مدیریت پرونده‌های جنایی است. وظایف اصلی این مؤلفه عبارتند از:

1. ایجاد Lead جدید
 2. ویرایش Lead موجود
 3. حذف Lead
 4. مدیریت محتوای Lead:
 - یادداشت کارآگاهی (Detective Note)
 - شواهد (Evidence) مرتبط با پرونده
 5. تعیین موقعیت مکانی Lead در داشبورد (Position X و Y برای نمایش در رابط کاربری گرافیکی)
- این مؤلفه به DetectiveBoard متصل است و با آن از طریق callback های onSave , onDelete و onClose تعامل دارد.

2. Props

مؤلفه LeadModal سه prop اصلی دریافت می‌کند:

Prop	نوع	توضیح	
lead	Object	null	داده‌های Lead موجود برای ویرایش؛ اگر null باشد، مودال برای ایجاد Lead جدید است.
onClose	Function	تابعی برای بستن مودال.	
onSave	Function	تابع callback برای ذخیره یا بروزرسانی Lead.	
onDelete	Function	null	تابع callback برای حذف Lead؛ فقط در حالت ویرایش فعال است.

3. State داخلی

1. formData → شامل اطلاعات فرم

- `title` → عنوان Lead
 - `lead_type` → نوع Lead: N (Note) یا E (Evidence)
 - `content` → متن یادداشت برای Note نوع Lead
 - `evidence` → Evidence نوع Lead برای Evidence شناسه
 - `position_x` و `position_y` → Lead موقعیت مکانی (0 تا 1)
2. **evidences** → Evidence نوع Lead آرایه‌ای از شواهد موجود برای انتخاب در
3. **loading** → برای جلوگیری از چند بار کلیک Lead وضعیت ذخیره‌سازی
-

4. رفتار مؤلفه و چرخه حیات

4-1. useEffect

- وقتی `lead` تغییر کند:
- مقادیر فرم (`formData`) با داده‌های Lead پر می‌شوند.
- برای بارگذاری لیست شواهد موجود فراخوانی می‌شود `fetchEvidences()`.

4-2. fetchEvidences

- درخواست GET به `/api/evidences/evidences/` برای دریافت تمام شواهد موجود
- افزودن هدر `Authorization` با `Token` ذخیره‌شده
- ذخیره داده‌ها در `state evidences`

4-3. handleSubmit

- جلوگیری از رفتار پیش‌فرض فرم
 - فعال کردن حالت `loading`
 - فراخوانی `onSave(formData)` callback
 - پس از اتمام ذخیره، `loading` غیرفعال می‌شود
-

5. طراحی رابط کاربری

5-1. Overlay و Modal

- **overlay:** برای تاکید روی مودال (`rgba(0,0,0,0.7)`) پس‌زمینه نیمه شفاف مشکی
- **modal:** جعبه اصلی
- رنگ پس‌زمینه تیره (`1a1e24#`)
- حاشیه و گوشه‌های گرد

- سایز محدود (maxWidth: 90% ، حداکثر 500px)
- متن و عناصر فرم به رنگ روشن برای کنتراست

5-2. Header

- عنوان مودال: "New Lead" یا "Edit Lead"
- دکمه بستن × در سمت راست

5-3. فرم

1. **Title** → فیلد متنی
2. **Type** → انتخاب نوع (Evidence یا Detective Note)
3. **Content / Evidence** → نمایش شرطی بر اساس lead_type :
 - اگر textarea → N برای متن یادداشت
 - اگر select → E برای انتخاب Evidence موجود
4. **Position X/Y** → فیلد عددی بین 0 و 1

5. دکمه‌ها:

- **Delete** → فقط در حالت ویرایش
- **Cancel** → بستن مودال
- **Save** → ذخیره Lead (loading غیرفعال هنگام)

5-4. استایل‌ها

- طراحی مدرن و تاریک (Dark Mode) با رنگ‌های متضاد: #1a1e24, #2c3e50, #ecf0f1, #e0b84d
- ورودی‌ها و textarea با پس‌زمینه تیره و حاشیه روشن
- دکمه‌ها دارای hover و transition برای تجربه کاربری بهتر

6. تعامل با سایر مؤلفه‌ها و API

1. DetectiveBoard:

- نمایش Lead در پنل
- فراخوانی onSave و onDelete برای ایجاد/ویرایش/حذف Lead

2. API Endpoints (در مؤلفه والد onSave و onDelete از طریق):

- ایجاد /api/detection/leads → POST /Lead
- بروزرسانی /api/detection/leads/{id} → PUT /Lead
- حذف /api/detection/leads/{id} → DELETE /Lead

3. فراخوانی شواهد برای Lead نوع Evidence:

- GET /api/evidences/evidences/
-

7. نکات فنی

- Lead برای فرم بر اساس نوع Conditional Rendering
 - Validation ساده:
 - همه فیلدها required
 - بین 0.01 و 0.99 Position X/Y
 - State Management داخلی برای فرم
 - جلوگیری از propagation کلیک overlay برای جلوگیری از بسته شدن تصادفی مودال
 - قابلیت استفاده مجدد برای ایجاد و ویرایش Lead با یک مؤلفه
-

8. ارتباط با نیازمندی‌های پروژه

- مدیریت Lead یکی از بخش‌های کلیدی صفحه DetectiveBoard است
 - پشتیبانی از:
 - یادداشت کارآگاهی
 - شواهد متصل به پرونده
 - تعیین مکان Lead در داشبورد
 - کاربر می‌تواند Leadها را ایجاد، ویرایش و حذف کند
 - رعایت UX مناسب با مودال، overlay و دکمه‌های واضح
-

9. جمع‌بندی

مؤلفه LeadModal یک پنجره مودال کامل برای مدیریت Leadها ارائه می‌کند و:

1. تعامل امن با API و مدیریت state
2. طراحی مدرن تاریک با UX مناسب
3. پشتیبانی از ایجاد، ویرایش و حذف Lead
4. امکان انتخاب شواهد موجود برای Lead نوع Evidence

را فراهم می‌کند و مستقیماً با DetectiveBoard برای نمایش و مدیریت Leadها هماهنگ است.

گزارش فنی مؤلفه Login (فرانت‌اند - فرم ورود)

1. هدف و جایگاه مؤلفه در سامانه

مؤلفه Login یک فرم ورود کاربر است که به کاربران اجازه می‌دهد با نام کاربری، کد ملی، شماره تلفن یا ایمیل وارد سامانه مدیریت پرونده‌های جنایی شوند. وظایف اصلی این مؤلفه عبارتند از:

1. جمع‌آوری اطلاعات ورود کاربر (نام کاربری و رمز عبور)
2. ارسال درخواست احراز هویت به سرور
3. مدیریت وضعیت Loading و خطا
4. ذخیره توکن احراز هویت و اطلاعات کاربر در کانتکست AuthContext
5. هدایت کاربر به داشبورد پس از ورود موفق

این مؤلفه به عنوان صفحه `login/` در مسیرهای فرانت‌اند عمل می‌کند و با AuthContext برای مدیریت وضعیت کاربر هماهنگ است.

2. استفاده شده Context و Props

- **Props:** خارجی دریافت نمی‌کند و کاملاً مستقل است prop این مؤلفه هیچ
- **Context:**
 - `useAuth()` :
 - `login(token, user)` → Context ذخیره توکن و اطلاعات کاربر در `localStorage`
- **Router:**
 - `useNavigate()` → هدایت به مسیر `/dashboard` پس از ورود موفق
 - `<Link to="/register">` → هدایت کاربر به صفحه ثبت‌نام

3. State داخلی

State	نوع	توضیح	
<code>formData</code>	<code>{username, password}</code>	ذخیره مقادیر ورودی فرم	
<code>loading</code>	<code>boolean</code>	وضعیت ارسال درخواست؛ برای غیرفعال کردن دکمه Login و نمایش متن "...Logging in"	
<code>error</code>	<code>string</code>	<code>null</code>	نمایش پیام خطا در صورت ناموفق بودن ورود

4. رفتار مؤلفه

4-1. handleChange

- بروزرسانی `formData` هنگام تغییر مقدار `input`ها
- استفاده از `computed property` `[e.target.name]` برای هماهنگی با نام فیلد

4-2. handleSubmit

- جلوگیری از رفتار پیش فرض فرم (`e.preventDefault()`)
- فعال کردن `loading` و حذف پیام خطا
- ارسال درخواست POST به `endpoint` احراز هویت:

```
POST http://localhost:8000/api/token-auth/  
Headers: { "Content-Type": "application/json" }  
Body: { username, password }
```

- اگر پاسخ موفق (`res.ok`) باشد:
 1. دریافت توکن و اطلاعات کاربر از سرور
 2. فراخوانی `login(response.token, response.user)` برای ذخیره در `Context` و `localStorage`
 3. هدایت کاربر به مسیر `dashboard/`
- اگر پاسخ ناموفق باشد:
 - نمایش پیام خطا از سرور یا پیام پیش فرض `"Invalid username or password"`
 - در صورت خطای شبکه:
 - نمایش پیام `".Network error. Please check your connection"`
 - در نهایت، `loading` غیرفعال می شود

5. طراحی رابط کاربری

5-1. Container

- فرم با حداکثر عرض `350px` و `margin` مرکزی
- داخلی مناسب برای فاصله از لبه ها `padding`

5-2. فرم

- Input ها:

- نام کاربری و رمز عبور
 - Required و دارای placeholder
 - استایل ساده و مدرن با رنگ پس‌زمینه روشن
 - **Button Login:**
 - سبز و با متن "Login"
 - در حالت loading غیرفعال و رنگ خاکستری
 - پیغام خطا:
 - پس‌زمینه قرمز روشن، متن قرمز تیره، گوشه‌های گرد
 - ظاهر زیر دکمه Login
 - لینک ثبت‌نام:
 - متن با رنگ خاکستری
 - `<Link>` به مسیر `/register`
-

6. تعامل با API و سرور

- Endpoint اصلی: `POST /api/token-auth/`
- فرمت پاسخ `(LoginResponse)`:

```
interface LoginResponse {
  token: string; // توکن احراز هویت
  user: User;    // نقش‌ها و سایر مشخصات
}
```

- `AuthContext.login(token, user):`
 - ذخیره توکن در `localStorage`
 - ذخیره اطلاعات کاربر در `Context` برای استفاده در کل برنامه
-

7. نکات فنی

1. استفاده از `TypeScript` برای تعیین نوع پاسخ و متغیرها
 2. مدیریت حالت `Loading` برای جلوگیری از ارسال چندباره فرم
 3. مدیریت خطای شبکه و خطای سرور به صورت مجزا
 4. استفاده از `Router Navigate` برای هدایت کاربر بعد از ورود موفق
 5. طراحی ساده، قابل استفاده در حالت موبایل و دسکتاپ
-

8. ارتباط با نیازمندی‌های پروژه

- این مؤلفه امنیت ورود کاربر را مدیریت می‌کند
 - تضمین می‌کند که تنها کاربران احراز هویت شده بتوانند وارد داشبورد و سایر صفحات شوند
 - ارتباط مستقیم با AuthContext و مدیریت توکن، نقش‌ها و دسترسی‌ها دارد
-

9. جمع‌بندی

مؤلفه Login:

- رابط کاربری ساده و کاربرپسند برای ورود به سامانه فراهم می‌کند
 - با API احراز هویت ارتباط برقرار می‌کند و توکن را ذخیره می‌کند
 - پیام خطا و وضعیت Loading را مدیریت می‌کند
 - پس از ورود موفق، کاربر به داشبورد هدایت می‌شود
 - نقش کلیدی در کنترل دسترسی کاربران در سامانه دارد
-

گزارش فنی مؤلفه ModulesRegistry (فرانت‌اند - مدیریت ماژول‌ها)

1. هدف و جایگاه مؤلفه

مؤلفه ModulesRegistry مسئول مدیریت نمایش ماژول‌های فرانت‌اند و ارائه داشبورد پویا بر اساس دسترسی‌های کاربر است.
این مؤلفه:

1. نمایش ماژول‌های قابل دسترس بر اساس مجوزهای (Permissions) کاربر
2. مدیریت نمایش ماژول به صورت تمام صفحه
3. ارائه دکمه‌های اقدامات سریع (Quick Actions) بر اساس دسترسی‌ها
4. پشتیبانی از ناوبری بین داشبورد و ماژول‌ها

این مؤلفه در واقع صفحه اصلی داشبورد کاربران پس از ورود به سامانه است و رابط بین ماژول‌ها و داشبورد را فراهم می‌کند.

2. Props

توضیح	نوع	Prop
اطلاعات کاربر (نام، نام کامل، نقش‌ها و ...)	object	user
آرایه‌ای از مجوزهای کاربر برای نمایش و استفاده از ماژول‌ها	string[]	permissions

3. State داخلی

توضیح	نوع	State
ماژول فعال که کاربر در حال مشاهده آن است؛ اگر null باشد داشبورد نمایش داده می‌شود	object	activeModule
پراپ‌های اضافی که برای ماژول فعال ارسال می‌شود	object	moduleProps

4. پیکربندی ماژول‌ها (MODULE_CONFIG)

هر ماژول در این بخش تعریف شده و شامل مشخصات زیر است:

Grid Size	Priority	Icon	Title	Component	Permission Key
large	1		Active Cases	CasesModule	"cases.view"
medium	2		Evidence Locker	EvidenceModule	"evidence.view"
medium	3		Interrogations	InterrogationModule	"interrogation.view"
small	4		Reports	ReportsModule	"reports.view"

- **priority:** ترتیب نمایش ماژول‌ها در داشبورد
- **gridSize:** (large, medium, small) اندازه کارت ماژول در شبکه

5. مدیریت دسترسی‌ها

- ابتدا تمام ماژول‌ها از MODULE_CONFIG استخراج می‌شوند
- سپس با استفاده از permissions کاربر، فقط ماژول‌های قابل دسترس فیلتر می‌شوند:

```
availableModules = Object.entries(MODULE_CONFIG)
  .filter(([permission]) => permissions.includes(permission))
  .map([_, config]) => config
  .sort((a,b) => a.priority - b.priority)
```

- به این ترتیب داشبورد هر کاربر دینامیک و بر اساس دسترسی‌ها ساخته می‌شود.

6. رفتار مؤلفه

6-1. انتخاب ماژول

- هنگام کلیک روی کارت ماژول:

```
handleModuleSelect(module, props)
```

- مقدار `activeModule` به ماژول انتخابی تغییر می‌کند
- نمایش ماژول به حالت تمام صفحه تغییر می‌کند

6-2. بازگشت به داشبورد

- با کلیک روی دکمه "Back to Dashboard" در ماژول فعال:

```
handleBackToDashboard()
```

- مقدار `activeModule` به `null` بازمی‌گردد
- داشبورد با کارت‌های ماژول نمایش داده می‌شود

7. طراحی رابط کاربری

7-1. داشبورد

1. Welcome Section:

- نمایش نام کاربر و تعداد ماژول‌های قابل دسترس

2. Modules Grid:

- شبکه‌ای از کارت‌های ماژول با آیکون، عنوان و توضیح کوتاه
- کارت‌ها با سایه و افکت `hover`
- اندازه کارت‌ها بر اساس `gridSize` (large, medium, small)

3. Quick Actions:

- دکمه‌های سریع مانند "New Case", "Log Evidence", "Generate Report"
- تنها در صورتی که کاربر مجوز ایجاد آن مورد را داشته باشد نمایش داده می‌شوند

7-2. ماژول فعال (Fullscreen Module)

- نمایش ماژول انتخابی به صورت تمام صفحه با هدر اختصاصی
 - هدر شامل:
 - دکمه بازگشت
 - عنوان و آیکون ماژول
 - محتوا: رندر کامپوننت ماژول انتخابی با پراپ‌های کاربر و permissions
-

8. نکات فنی

1. استفاده از React useState برای مدیریت حالت فعال ماژول و پراپ‌های اضافی
 2. ماژول‌ها بر اساس دسترسی و اولویت Sort و Filter
 3. کد سبک و قابل توسعه برای اضافه کردن ماژول‌های جدید تنها با افزودن به MODULE_CONFIG
 4. رندر داینامیک ماژول‌ها بدون نیاز به تغییر ساختار اصلی داشبورد
 5. مدیریت ناوبری بین داشبورد و ماژول فعال به صورت ساده
-

9. ارتباط با نیازمندی‌های پروژه

- ارائه داشبورد مرکزی برای کاربر با دسترسی‌های تعریف شده
 - اطمینان از اینکه کاربر فقط ماژول‌هایی که اجازه دارد مشاهده می‌کند
 - پشتیبانی از اقدامات سریع بر اساس مجوز
 - نمایش ماژول‌ها در ابعاد و ترتیب مناسب برای تجربه کاربری بهتر
-

10. جمع‌بندی

مؤلفه ModulesRegistry:

- رابط کاربری داشبورد و ماژول‌ها را مدیریت می‌کند
- دسترسی‌ها و اولویت‌ها را برای نمایش ماژول‌ها اعمال می‌کند
- امکان باز کردن هر ماژول به صورت تمام صفحه و بازگشت به داشبورد را فراهم می‌کند
- ماژول‌ها و اقدامات سریع با توجه به permissions کاربر نمایش داده می‌شوند

- قابل توسعه برای افزودن ماژول‌های جدید با حداقل تغییرات

گزارش فنی مؤلفه PreComplaintView (فرانت‌اند – پیش‌نمایش شکایات)

1. هدف و جایگاه مؤلفه

مؤلفه PreComplaintView مسئول نمایش پیش‌نمایش شکایات‌ها و مدیریت عملیات مرتبط با آن‌ها در داشبورد سامانه پلیس است.
این مؤلفه:

1. نمایش جزئیات شکایت (Title, Status, Crime Level, نقش کاربر)
 2. مدیریت تب‌ها برای مشاهده بخش‌های مختلف شکایت:
 - جزئیات شکایت (Details)
 - شاکیان (Complainants)
 - بررسی‌ها (Reviews)
 - تاریخچه (Timeline)
 - مدارک (Documents)
 3. نمایش دکمه‌های عملیاتی بر اساس نقش کاربر و وضعیت شکایت
 4. مدیریت ارسال برای بررسی، ویرایش، ارجاع به افسر، تأیید/رد نهایی، درخواست اطلاعات بیشتر، حذف و ارتقا سطح شکایت
 5. پشتیبانی از مدال بررسی (Review Modal) برای نقش‌های Supervisor و Cadet, Officer, Admin
- این مؤلفه به عنوان صفحه پیش‌نمایش شکایت قبل از اقدامات نهایی در فرانت‌اند عمل می‌کند.

2. Props

Prop	نوع	توضیح				
complaint	RegisterComplain	داده کامل شکایت برای نمایش				
onEdit	() => void	تابع ویرایش شکایت				

Prop	نوع	توضیح				
onSubmit	() => void	تابع ارسال شکایت برای بررسی				
onCancel	() => void	تابع لغو شکایت				
onDelete	() => void	تابع حذف شکایت				
onReview	(review: Partial) => void	تابع ثبت بررسی برای شکایت				
onAssignToOfficer	() => void	تابع ارجاع شکایت به افسر				
onRequestMoreInfo	() => void	تابع درخواست اطلاعات بیشتر				
onEscalate	() => void	تابع ارتقا شکایت به مقام بالاتر				
currentUser	User	اطلاعات کاربر جاری				
userRole	'complainant'	'cadet'	'officer'	'admin'	'supervisor'	ناربر گیری
isLoading	boolean (اختیاری)	وضعیت بارگذاری محتوا				

3. State داخلی

State	نوع	توضیح				
activeTab	'details'	'complainants'	'reviews'	'timeline'	'documents'	فعال ، ش نمایش کایت
reviewMessage	string	پیام متنی بررسی وارد شده توسط کاربر				
reviewAction	'approve'	'reject'	'return'	null	نوع اقدام بررسی (تأیید، رد، بازگشت)	
showReviewModal	boolean	نمایش یا عدم نمایش مدال بررسی				

4. مدیریت رنگ ها و وضعیت ها

1. Badge وضعیت شکایت (Status Badge)

- هر وضعیت شکایت (ComplainStatus) رنگ و سبک متفاوتی دارد (Draft, Pending Cadet, Pending Officer, Returned, Approved, Rejected, Cancelled)

2. سطح جرم (CrimeType)

- نمایش سطح جرم با رنگ بندی متفاوت:
- سطح ۳: جرائم خرد - سبز
- سطح ۲: جرائم متوسط - زرد
- سطح ۱: جرائم سنگین - نارنجی
- بحرانی: قرمز

3. فرمت تاریخ

- تمامی تاریخ ها به فرمت فارسی (fa-IR) نمایش داده می شوند با ساعت و دقیقه.

5. مدیریت دکمه های عملیاتی بر اساس نقش

نقش کاربر	اقدامات مجاز
Complainant	ویرایش شکایت، ارسال برای بررسی، لغو شکایت (در وضعیت Draft)

نقش کاربر	اقدامات مجاز
Cadet	بررسی شکایت، ارجاع به افسر، درخواست اطلاعات بیشتر
Officer	تأیید نهایی، رد یا ارتقا شکایت (Escalate)
Supervisor	بازبینی، تخصیص به افسر، ارتقا سطح شکایت، حذف
Admin	دسترسی کامل به ویرایش، بررسی، تخصیص، حذف

- دکمه‌ها دارای آیکون، رنگ و افکت `hover` و `focus` هستند
- نمایش دکمه‌ها به صورت شرطی بر اساس `userRole` و وضعیت شکایت (`complaint.status`)

6. تب‌ها (Tabs)

- **Details:** نمایش جزئیات کامل شکایت
- **Complainants:** لیست شاکیان و اطلاعات آن‌ها
- **Reviews:** Cadet، Officer یا Admin بررسی‌های ثبت شده توسط
- **Timeline:** تاریخچه تغییر وضعیت شکایت
- **Documents:** مدارک و شواهد ثبت شده برای شکایت
- تب فعال در `activeTab` State نگهداری می‌شود
- کلید تب‌ها برای تغییر `activeTab` استفاده می‌شود

7. مدال بررسی (Review Modal)

- برای نقش‌های Cadet، Officer و Admin
- امکان وارد کردن پیام و انتخاب اقدام (`approve` , `reject` , `return`)
- باز و بسته شدن مدال با `showReviewModal` State کنترل می‌شود

8. ویژگی‌های فنی

1. استفاده از `React Functional Component` و `Hooks (useState)`
2. **RTL Support:** جهت راست به چپ برای فارسی
3. مدیریت بارگذاری محتوا با `isLoading` و انیمیشن چرخشی
4. رندر داینامیک دکمه‌ها و تب‌ها بر اساس نقش و وضعیت شکایت
5. استفاده از `TailwindCSS` برای طراحی رابط کاربری واکنش‌گرا و مدرن
6. **Extensible:** افزودن نقش‌ها، اقدامات یا تب‌های جدید با تغییر محدود در کامپوننت

9. ارتباط با نیازمندی‌های پروژه

- نمایش جزئیات کامل شکایت قبل از ارسال و بررسی
- اطمینان از اینکه کاربران فقط عملیات مجاز خود را می‌توانند انجام دهند
- ارائه روند بازبینی شکایت برای Officer و Cadet
- امکان ارجاع، ارتقا، حذف و درخواست اطلاعات بیشتر
- پشتیبانی از چند نقش کاربری با سطوح دسترسی متفاوت

10. جمع‌بندی

مؤلفه PreComplaintView یک ابزار جامع و واکنش‌گرا برای پیش‌نمایش و مدیریت شکایات است که:

- رابط کاربری غنی با تب‌های مختلف و مدال بررسی ارائه می‌دهد
- عملکرد عملیاتی را بر اساس نقش و وضعیت شکایت محدود و کنترل می‌کند
- با TailwindCSS و React Hooks طراحی شده و قابل توسعه برای افزودن نقش‌ها، تب‌ها و اقدامات جدید است
- محوریّت آن امنیت و کنترل دسترسی‌ها در سطح فرانت‌اند است

گزارش فنی مؤلفه Register (فرانت‌اند - ثبت‌نام کاربر)

1. هدف و جایگاه مؤلفه

مؤلفه Register مسئول فرم ثبت‌نام کاربران جدید در سامانه است. این فرم امکان دریافت اطلاعات هویتی و ورود کاربران را فراهم می‌کند و پس از ثبت‌نام، کاربر را وارد سیستم کرده و به داشبورد هدایت می‌کند.

این مؤلفه وظایف زیر را انجام می‌دهد:

1. نمایش فرم ثبت‌نام با فیلدهای مورد نیاز
2. مدیریت State فرم و تغییرات آن در زمان واقعی
3. ارسال اطلاعات به سرور با متد POST و پردازش پاسخ آن
4. مدیریت وضعیت بارگذاری (Loading) و نمایش پیام خطا
5. ورود خودکار کاربر به سیستم پس از ثبت‌نام موفق

2. داخلی State

توضیح	نوع	State
نگهداری داده‌های فرم شامل <code>username</code> , <code>password</code> , <code>full_name</code> , <code>email</code> و <code>national_id</code> , <code>phone_number</code>	object	<code>formData</code>
وضعیت ارسال فرم و نشان دادن دکمه در حالت غیرفعال و پیام "Creating Account..."	boolean	<code>loading</code>
	string	<code>error</code> null

3. مدیریت تغییرات فرم

- تابع `handleChange` برای به‌روزرسانی `State` فرم هنگام تغییر در هر فیلد استفاده می‌شود.
- از `e.target.name` و `e.target.value` برای نگهداری مقدار هر فیلد در `formData` استفاده شده است.

4. ارسال فرم و ارتباط با سرور

- تابع `handleSubmit` هنگام ارسال فرم اجرا می‌شود:

فرآیند کلی:

1. جلوگیری از رفتار پیش‌فرض فرم (`e.preventDefault()`)
2. فعال کردن حالت بارگذاری و پاک کردن خطاهای قبلی
3. ارسال درخواست **POST** به `http://localhost:8000/api/register` با `Content-Type: application/json` و بدنه `formData`
4. بررسی پاسخ سرور:
 - اگر `res.ok` باشد:
 - کاربر با `login(data.token, data.user)` وارد سیستم می‌شود
 - هدایت کاربر به `dashboard/`
 - اگر پاسخ خطا داشته باشد:
 - بررسی خطاهای فیلدهای `username`, `email`, `national_id`
 - نمایش پیام مناسب خطا
 - اگر خطای شبکه رخ دهد، پیام `Network error` نمایش داده می‌شود

نکات مهم:

- خطاهای سرور به صورت پیام‌های خوانا برای کاربر نمایش داده می‌شوند.

- مدیریت استثنا با try-catch و استفاده از finally برای غیرفعال کردن حالت Loading انجام شده است.

5. استایل و رابط کاربری

- تمام استایل‌ها با استفاده از inline styles در آبجکت styles مدیریت شده‌اند.
- مؤلفه طراحی واکنش‌گرا و ساده دارد و شامل بخش‌های زیر است:
 - خودکار برای مرکزیت صفحه margin , maxWidth: 400px : فرم Container
 - Form: padding, border, border-radius, background سفید
 - Input fields: padding, border, border-radius, فونت مناسب
 - Button :ثبت نام
- رنگ آبی، فونت برجسته، radius مناسب، حالت hover و disabled
- پیام خطا: رنگ پس‌زمینه قرمز روشن با متن قرمز تیره و متن وسط چین
- لینک ورود: متن خاکستری و مرکز چین

6. فیلدهای فرم

فیلد	نوع	الزامی
Username	text	بله
Password	password	بله
Full Name	text	بله
National ID	text	بله
Phone Number	text	بله
Email	email	بله

7. ویژگی‌های فنی

1. React Functional Component با Hooks (useState)
2. React Router Integration:
 - برای هدایت کاربر به داشبورد useNavigate
 - برای لینک به صفحه ورود Link
3. Context API (useAuth) برای مدیریت ورود کاربر بعد از ثبت نام

4. مدیریت وضعیت فرم و خطاها به صورت داینامیک

5. استفاده از `async/await` برای درخواست HTTP

6. `Inline Styling` برای کنترل دقیق ظاهر فرم

8. امنیت و اعتبارسنجی

- تمام فیلدها با `required` مشخص شده‌اند
 - نوع ورودی برای ایمیل و رمز عبور مشخص شده است
 - خطاهای سرور برای جلوگیری از اطلاعات نامعتبر مدیریت شده‌اند
-

9. ارتباط با نیازمندی‌های پروژه

- فرم ثبت‌نام جدید برای ایجاد حساب کاربری در سیستم
 - ورود خودکار کاربر پس از ثبت‌نام موفق و هدایت به داشبورد
 - نمایش پیام خطا در صورت نامعتبر بودن داده‌ها
 - رابط کاربری ساده و واکنش‌گرا
-

10. جمع‌بندی

مؤلفه `Register` یک فرم کامل ثبت‌نام است که:

- تمامی فیلدهای هویتی مورد نیاز پروژه را دریافت می‌کند
 - خطاهای سرور و شبکه را مدیریت می‌کند
 - پس از موفقیت، کاربر را وارد سیستم کرده و به داشبورد هدایت می‌کند
 - از `React Hooks` و `Context` برای مدیریت `State` و ورود کاربر استفاده می‌کند
 - طراحی ساده و قابل توسعه دارد و می‌تواند به راحتی با استایل‌های پیشرفته‌تر یا اعتبارسنجی‌های بیشتر بهبود یابد
-

گزارش فنی مؤلفه `SuspectSuggestionModal` (فرانت‌اند – پیشنهاد مظنون)

1. هدف و جایگاه مؤلفه

مؤلفه `SuspectSuggestionModal` یک پنجره مودال (Modal) است که برای نمایش امکان پیشنهاد مظنونین برای یک پرونده یا تخته (board) طراحی شده است.

در حال حاضر، این ویژگی هنوز در حالت توسعه یا **Coming Soon** قرار دارد و عملکرد کامل آن برای ارسال پیشنهاد مظنونین به سرور یا ذخیره سازی، پیاده سازی نشده است.

این مؤلفه تنها:

1. نمایش مودال با عنوان "Suggest Suspects"
2. ارائه پیام اطلاع رسانی "This feature is coming soon..."
3. ارائه دکمه بستن مودال

2. Props مؤلفه

توضیح	نوع	Prop
	string	boardId
تابع callback برای بستن مودال	function	onClose
تابع callback برای موفقیت عملیات پیشنهاد مظنون (در نسخه فعلی استفاده نشده)	function	onSuccess

نکته: در نسخه فعلی فقط `onClose` استفاده شده و `onSuccess` و `boardId` آماده استفاده در توسعه های بعدی هستند.

3. ساختار داخلی مؤلفه

- لایه کلی (Overlay):
 - ویژگی ها: `position: fixed`, `top:0`, `left:0`, `right:0`, `bottom:0`
 - پس زمینه نیمه شفاف با `rgba(0,0,0,0.5)`
 - مرکز چین محتوا با `display: flex` و `justify-content: center`, `align-items: center`
- مودال (Modal Box):
 - `div` با `background-color: white`, `padding: 20px`, `border-radius: 8px`
 - حداکثر عرض `500px` و عرض نسبی `90%` برای واکنش گرایی
- محتوای مودال:
 - عنوان `h2` با متن "Suggest Suspects"

- پاراگراف اطلاع رسانی با متن "This feature is coming soon..."
 - دکمه بستن با استایل ساده و قابل کلیک
-

4. استایل‌ها

تمام استایل‌ها با `inline styles` مدیریت شده‌اند:

- Overlay: پس‌زمینه نیمه‌شفاف و مرکزچین محتوا
 - Box: گوشه‌های گرد و محدودیت حداکثر عرض، `padding`، رنگ سفید
 - دکمه Close: رنگ آبی (`#3498db`)، متن سفید، گوشه‌های گرد، `cursor pointer`، فاصله از محتوا
-

5. رفتار و تعاملات

- باز و بسته شدن مودال:
 - باز شدن: مؤلفه زمانی که در DOM رندر شود، به صورت خودکار نمایش داده می‌شود
 - بسته شدن: با کلیک روی دکمه Close، تابع `onClose` فراخوانی می‌شود
 - تعامل با سایر توابع:
 - `callback`، برای توسعه‌های بعدی آماده شده تا بتواند پس از ثبت مپنون `onSuccess` Props اجرا شود
 - برای شناسایی تخته/پرونده در توسعه‌های بعدی کاربرد دارد `boardId` Props
-

6. نقاط قابل توسعه

1. ارسال داده به سرور:
 - انتخاب مپنون و ارسال پیشنهاد با استفاده از `boardId` و `onSuccess`
 2. لیست مپنونین:
 - نمایش یک فرم یا لیست انتخاب چندمپنونی
 3. اعتبارسنجی و پیام موفقیت/خطا:
 - نمایش پیام مناسب بعد از موفقیت یا خطا
 4. استایل پیشرفته‌تر:
 - استفاده از Tailwind یا CSS Modules برای هماهنگی با سایر مودال‌ها
-

7. ارتباط با نیازمندی‌های پروژه

- این مؤلفه بخشی از سیستم مدیریت پرونده‌ها و مظنونین است.
- در حال حاضر یک پیش‌نمایش یا مودال اطلاع‌رسانی ارائه می‌دهد.
- آماده استفاده و گسترش برای مراحل بعدی است که شامل پیشنهاد مظنونین واقعی و ذخیره‌سازی می‌باشد.

8. جمع‌بندی

مؤلفه `SuspectSuggestionModal` :

- یک مودال ساده و واکنش‌گرا است که قابلیت باز و بسته شدن دارد
- در وضعیت فعلی، عملکرد نهایی ثبت مظنون را پیاده نکرده است
- آماده استفاده در توسعه‌های بعدی هستند `onSuccess` و `Props boardId`
- رابط کاربری ساده و قابل فهم، مناسب برای اطلاع‌رسانی و توسعه آینده

گزارش فنی مؤلفه YarnModal (فرانت‌اند - اتصال سرخ‌ها)

1. هدف و جایگاه مؤلفه

مؤلفه `YarnModal` یک پنجره مودال (Modal) برای اتصال دو سرخ (Lead) به یکدیگر در سیستم مدیریت پرونده‌ها و تخته‌ها است.

هدف اصلی این مؤلفه:

1. انتخاب دو سرخ از میان لیست موجود (`leads`)
2. ایجاد اتصال یا ارتباط بین آن‌ها
3. ارسال داده‌های انتخاب شده به یک تابع (`onSave`) callback برای پردازش بیشتر

2. Props مؤلفه

توضیح	نوع	Prop
	string	<code>boardId</code>
آرایه‌ای از سرخ‌ها با ساختار <code>{id, title, lead_type}</code> برای نمایش در لیست انتخاب	array	<code>leads</code>
تابع callback برای بستن مودال	function	<code>onClose</code>

توضیح	نوع	Prop
تابع callback برای ذخیره یا پردازش اتصال سرخ‌ها، داده‌ها به شکل {lead1, lead2} به آن ارسال می‌شود	function	onSave

3. ساختار داخلی مؤلفه

3.1 State داخلی

توضیح	نوع	State
شناسه سرخ اول انتخاب شده	string	lead1
شناسه سرخ دوم انتخاب شده	string	lead2
وضعیت در حال بارگذاری (زمانی که onSave فراخوانی می‌شود)	boolean	loading

3.2 Overlay

- اصلی با ویژگی‌های `div`:
 - پوشش کامل صفحه ، `position: fixed`
 - برای نیمه‌شفاف شدن پس‌زمینه `background-color: rgba(0,0,0,0.7)`
 - مرکزچین محتوا با `display: flex` و `justify-content: center` , `align-items: center`

3.3 Modal Box

- پس‌زمینه تیره (`1a1e24#`) با حاشیه روشن (`2c3e50#`)
- گوشه‌های گرد، padding داخلی و محدودیت عرض (`width: 400px` , `maxWidth: 90%`)
- رنگ متن روشن (`ecf0f1#`)
- دارای Header با عنوان و دکمه Close

3.4 فرم انتخاب سرخ‌ها

- دو فیلد `select` برای انتخاب Lead اول و دوم
- لیست گزینه‌ها بر اساس `prop leads` ساخته می‌شود
- نمایش نوع سرخ: "Evidence" برای `lead_type === "E"` و "Note" برای بقیه
- انتخاب هر دو سرخ الزامی (`required`)
- جلوگیری از انتخاب دو سرخ یکسان (`disabled` در دکمه Connect زمانی که `lead1 === lead2`)

3.5 دکمه‌ها

دکمه	رفتار
Cancel	فراخوانی <code>onClose</code> برای بستن مودال
Connect	فراخوانی <code>handleSubmit</code> و ارسال <code>{lead1, lead2}</code> به <code>onSave</code>

4. استایل‌ها

تمام استایل‌ها با `inline styles` تعریف شده‌اند:

- Overlay: پس‌زمینه تیره نیمه‌شفاف، مرکزچین محتوا
- Modal: و رنگ متن روشن `padding`، پس‌زمینه تیره، حاشیه روشن، گوشه‌های گرد
- Header: جداکننده با خط پایین، عنوان با رنگ طلایی (#e0b84d)
- دکمه‌ها: تفاوت رنگ بین Cancel، Hover و Connect ساده
- Select: پس‌زمینه تیره، حاشیه روشن، متن روشن

5. رفتار و تعاملات

1. باز و بسته شدن مودال:

- کلیک روی پس‌زمینه یا دکمه Close → فراخوانی `onClose`

2. انتخاب سرنخ‌ها:

- کاربر دو سرنخ را از لیست کشویی انتخاب می‌کند
- انتخاب سرنخ یکسان غیرفعال است

3. ارسال داده‌ها:

- با کلیک روی Connect و پس از اعتبارسنجی، `handleSubmit` فراخوانی می‌شود
- داده‌های انتخاب شده به شکل `{lead1, lead2}` به `onSave` ارسال می‌شود
- وضعیت loading فعال می‌شود تا از ارسال چندباره جلوگیری شود

6. نقاط قابل توسعه

1. اعتبارسنجی پیشرفته‌تر:

- بررسی وجود سرنخ‌ها در سرور قبل از اتصال

2. نمایش پیام موفقیت یا خطا:

- اطلاع‌رسانی به کاربر بعد از اتصال سرنخ‌ها

3. بهبود رابط کاربری:

- اضافه کردن tooltip، رنگ‌بندی بر اساس نوع سرخ، یا نمایش توضیح کوتاه سرخ
-

7. ارتباط با نیازمندی‌های پروژه

- این مؤلفه بخش مدیریت ارتباط سرخ‌ها در تخته‌ها را پیاده‌سازی می‌کند
 - امکان اتصال دو Lead و ثبت این اتصال در backend فراهم شده است
 - کنترل انتخاب سرخ‌ها و جلوگیری از انتخاب یکسان مطابق با نیازمندی‌های عملکردی است
-

8. جمع‌بندی

مؤلفه YarnModal :

- یک مودال واکنش‌گرا برای اتصال دو Lead ارائه می‌دهد
 - اعتبارسنجی اولیه: انتخاب دو سرخ متفاوت و الزامی
 - مدیریت وضعیت loading در زمان ذخیره‌سازی
 - آماده توسعه بیشتر برای نمایش پیام موفقیت/خطا و تعامل با backend
-